Chapter 3: UI design, Intent, activity and service

Part I: Android App User Interface Design

Content

User interface

- **Android Layout, ViewGroup**
- **UI Element/ widget, View.**
- **Notification, Menus and Dialogs**
-

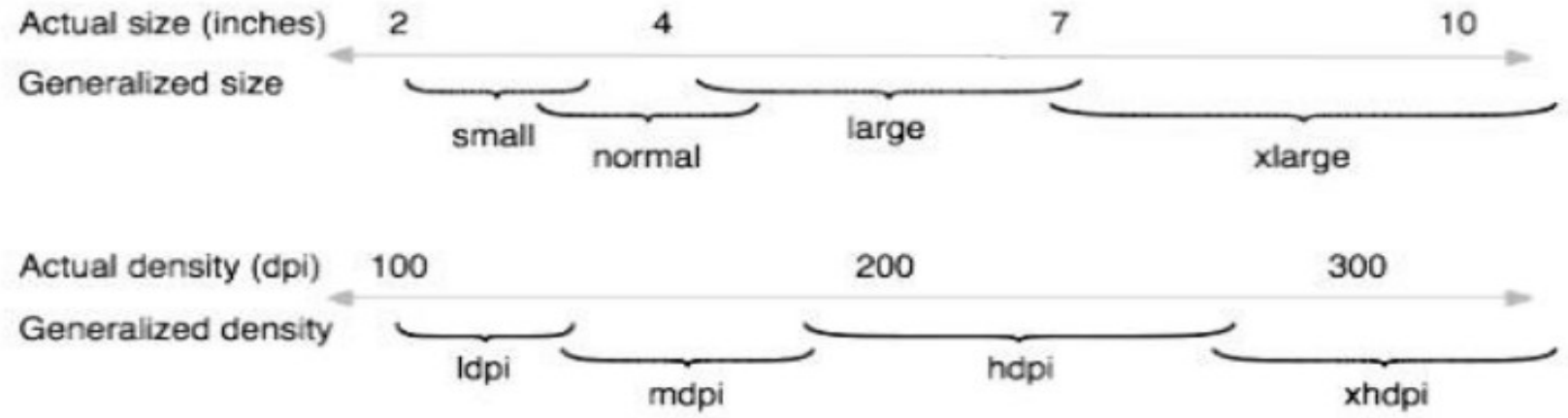
User Interface Layout

User interface Terminologies

- Orientation: The orientation of the screen from the user's point of view. There are two screen orientation:
  - Landscape orientation has a wide screen's aspect ratio while
  - Portrait orientation has a tall screen's aspect ratio
- different devices operate in different orientations by default, and the orientation can change at runtime when the user rotates the device.
- Resolution: The total number of physical pixels on a screen.
- Screen size is the Actual physical size, measured as the screen's diagonal. For simplicity, Android categorizes all actual screen sizes into four generalized sizes: small, normal, large, and extra large.

User interface Terminologies

- Screen density : The quantity of pixels within a physical area of the screen; usually referred to as dpi (dots per inch).
- Android categorizes screen densities to low, medium, high, and extra high.
- a "low" density screen has fewer pixels within a given physical area, compared to a "normal" or "high" density screen
- Illustration of how Android roughly maps actual screen sizes and screen densities to generalized sizes and densities (figures are not exact).



User interface Terminologies

- Density-independent pixel (dp): A virtual pixel unit that should be used when defining UI layout, to express layout dimensions or position in a density-independent way.
- The dp is equivalent to one physical pixel on a 160 dpi screen, which is the baseline density assumed by the system for a "medium" density screen.
- At runtime, the system transparently handles any scaling of the dp units, as necessary, based on the actual density of the screen in use.
- Android categorizes all density-independent pixel (dp) sizes into four generalized sizes: small, normal, large, and extra large.
  - xlarge screens are at least 960dp x 720dp
  - large screens are at least 640dp x 480dp
  - normal screens are at least 470dp x 320dp
  - small screens are at least 426dp x 320dp

User interface Terminologies

- The formula for converting dp units to screen pixels is as follows:
  - px = dp * (dpi / 160).
- For example, on a 240 dpi screen, 1 dp equals 1.5 physical pixels.
- dp units are used when defining application's UI, to ensure proper display of UI on screens with different densities.

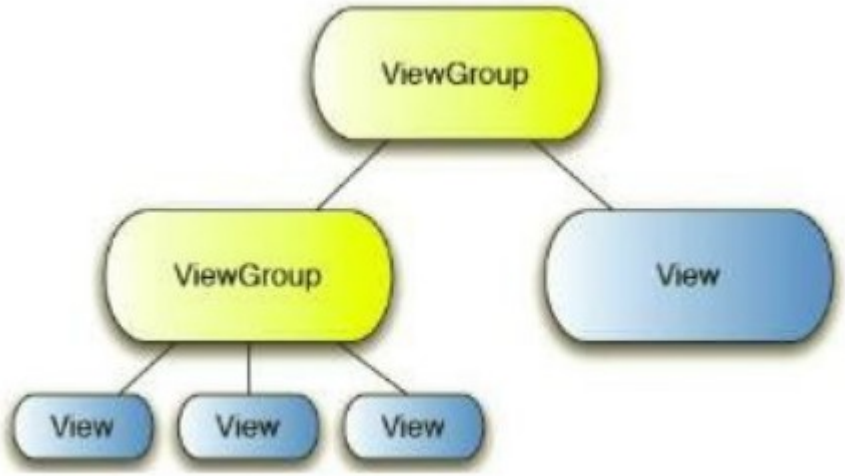
Units for specifying user interface

- The following table shows a list of units that are used to specify dimensions of user interface.

px (pixel)	Dots on the screen
in (inches)	Size as measured by a ruler
mm (millimeters)	Size as measured by a ruler
pt (points)	1/72 of an inch
dp (density-independent pixel)	Abstract unit. On screen with 160dpi, 1dp=1px
dip	synonym for dp and often used by Google

User Interface Layout

- The user interface layout of android application is defined using a hierarchy of Views and ViewGroups.
- In a single android user interface, Views and View groups are organized into a hierarchical tree structure.
- View groups are placed at the root and intermediate positions of the tree while Views are placed at leaf nodes of the tree.
- A single user interface can only have one root element.



Views

- A quick review on what a view is as covered from our previous lesson;
  - A View is a basic building block of UI (User Interface) in android.
  - A view is a small rectangular box that responds to user inputs. Eg: EditText, Button, CheckBox, etc.
  - View refer to the **android.view.View** class, which is the base class of all UI classes.
  - View is responsible for event handling and drawing.

ViewGroup

- ***So what about a View Group?***
  - A ***View Group*** is a subclass of the View class.
  - ***View Group*** acts as a base class for layouts and layouts parameters.
  - The ***View Group*** provides an invisible container to hold other Views or ***View Groups*** and to define the layout properties. For example, Linear Layout is the ***View Group*** that contains UI controls like Button, TextView, date picker etc., and other layouts also.
  - ***View Group*** Refer to the android.view.ViewGroup class, which is the base class of some special UI classes that can contain other View objects as children. Since ***View Group*** objects are also View objects, multiple ***View Group*** objects and View objects can be organized into an object tree to build a complex UI structure.

View Group Examples

- Examples of the commonly used ViewGroup subclasses used in android applications are;
  - FrameLayout
  - WebView
  - ListView
  - GridView
  - LinearLayout
  - RelativeLayout
  - TableLayout
- The above are generally termed as layouts. This will form our next item of discussion.
- But then, before we discuss about layouts, what is the difference between a view and viewgroup?

Differences between a view and a ViewGroup

View	*ViewGroup*
▪ View is a simple rectangle box that responds to the user's actions.	▪ View Group is the invisible container it holds views and viewGroup
▪ View is the Super Class of All component like TextView, EditText, ListView, etc	▪ ViewGroup is a collection of views(TextView, EditText, ListView, etc..), somewhat like a container.
▪ A View object is a component of the user interface (UI) like a button or text box, and it's also called a widget.	▪ A viewGroup object is a layout, that is, a container of other viewGroup objects(layouts) and view objects(widgets)
▪ Examples are EditText, Button, CheckBox, etc	▪ For example, LinerLayoutis the viewgroup that contains button(view), and other layouts also.
▪ View refers to the android view/ view class	▪ ViewGroup refers to the android view, viewGroup class
▪ Android.view.view which is the base class of all UI classes	▪ ViewGroup is the base class for layout.

Android Layouts

Android Layout

- It is used to define the user interface that holds the UI controls or widgets that will appear on the screen of an android application or activity screen.
- Generally, every application is a combination of View and ViewGroup.
- All the elements in a layout are built using a hierarchy of View and ViewGroup objects.
- A View usually draws something the user can see and interact with. Whereas a ViewGroup is an invisible container that defines the layout structure for View and other ViewGroup objects, as shown in below;

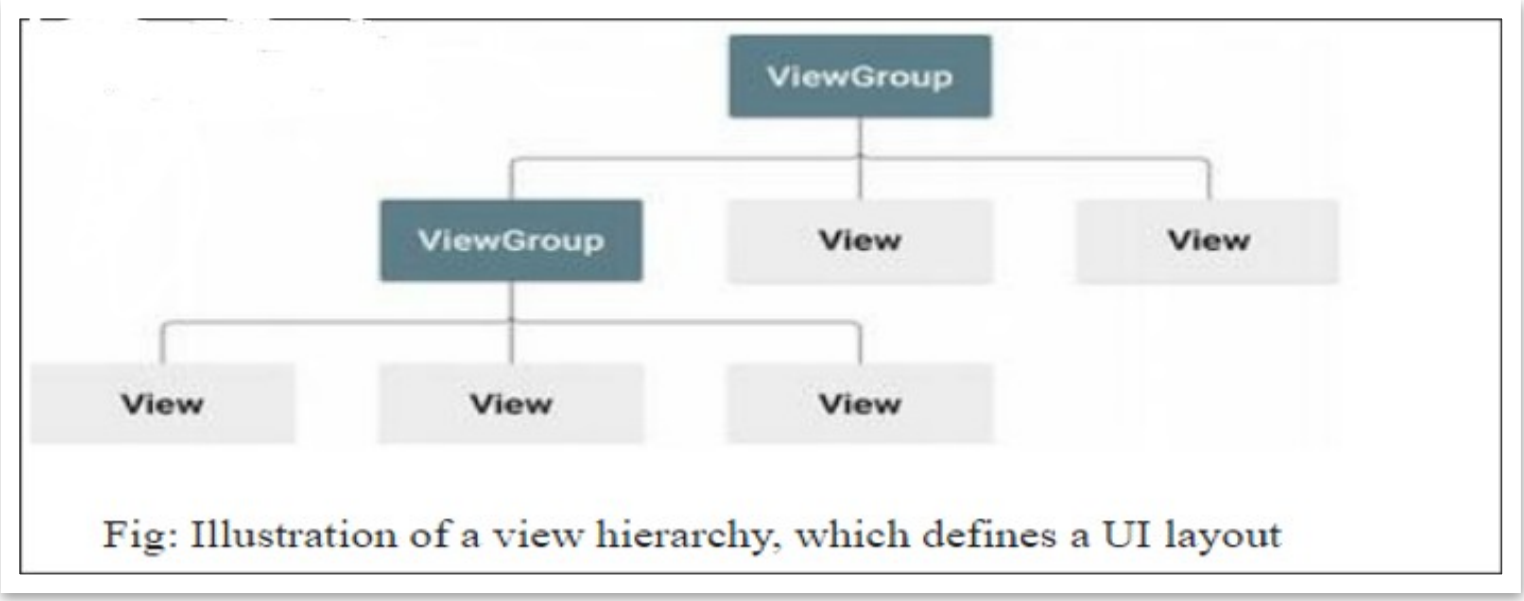


Fig: Illustration of a view hierarchy, which defines a UI layout

Android Layout

- The View objects are usually called "widgets" and can be one of many subclasses, such as Button or TextView.
- The ViewGroup objects are usually called "layouts".
- The ViewGroup can be one of many types that provide a different layout structure, such as LinearLayout or ConstraintLayout.
- How to declare a layout;
- You can declare a layout in several ways:

I. You can declare UI elements in XML. Android provides a straightforward XML vocabulary that corresponds to the View classes and subclasses, such as those for widgets and layouts. Declaring your UI in XML allows you to separate the presentation of your app from the code that controls its behavior. Using XML files also makes it easy to provide different layouts for different screen sizes and orientations.

Android Layout

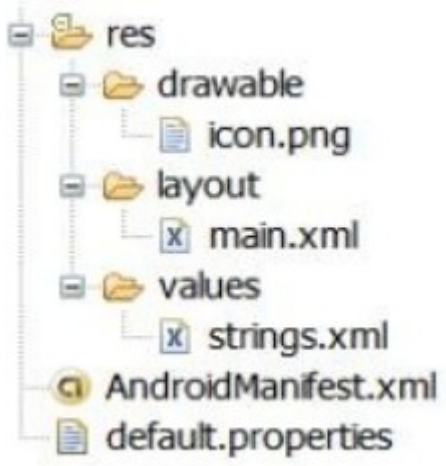
- The Android framework gives you the flexibility to use either or both of these methods to build your app's UI. For example, you can declare your app's default layouts in XML, and then modify the layout at runtime.

II. You can also use Android Studio's Layout Editor to build your XML layout using a drag-and-drop interface. The Layout Editor enables you to quickly build layouts by dragging UI elements into a visual design editor instead of writing layout XML by hand. The design editor can preview your layout on different Android devices and versions, and you can dynamically resize the layout to be sure it works well on different screen sizes.

III. You also instantiate layout elements at runtime. Your app can create View and ViewGroup objects (and manipulate their properties) programmatically.

Android Layout

- Layout resources are stored as XML files in the /res/layout resource directory for the application.



Android Layout Example

- The most common layouts provided by Android are :

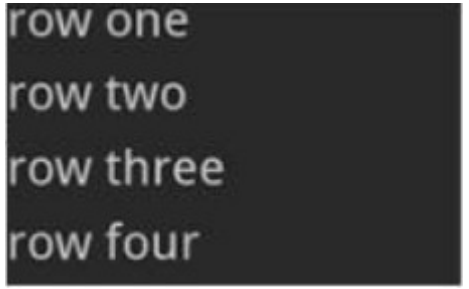
1. LinearLayout:
2. FrameLayout
3. RelativeLayout
4. TableLayout:
5. Constraint Layout

LinearLayout

- Linear layout arranges controls (its children) in a single column or single row. Controls can be aligned in one of the following direction.

1. Horizontally, this will only be one row high (the height of the tallest child, plus padding).

2. Vertically, this will only have one child per row, no matter how wide they are,

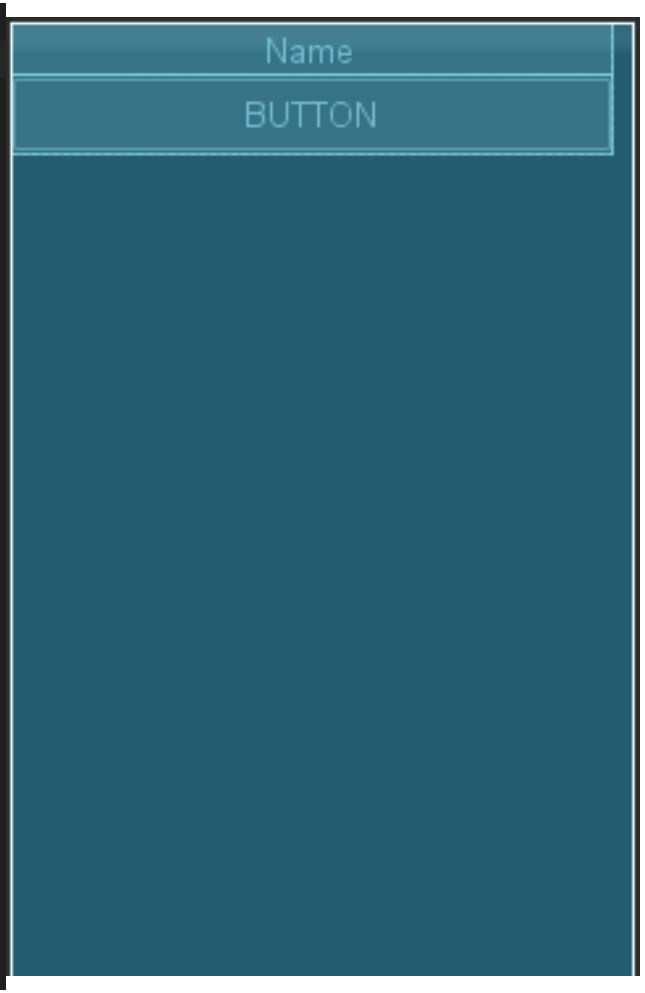


Linear layout is the most common layout

1. Linear Layout: XML File

- Example : The following XML code shows linear layout as the root view group with a text view and a button.

```xml
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:orientation="vertical"
    android:paddingRight="15sp"
    android:layout_height="match_parent">

    <TextView
        android:id="@+id/txtUsername"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="@string/userName"
        android:textAlignment="center"
        android:textSize="25sp" />

    <Button
        android:id="@+id/btnOk"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Button"
        android:textColor="@color/purple_700"
        android:textSize="25sp" />
</LinearLayout>
```

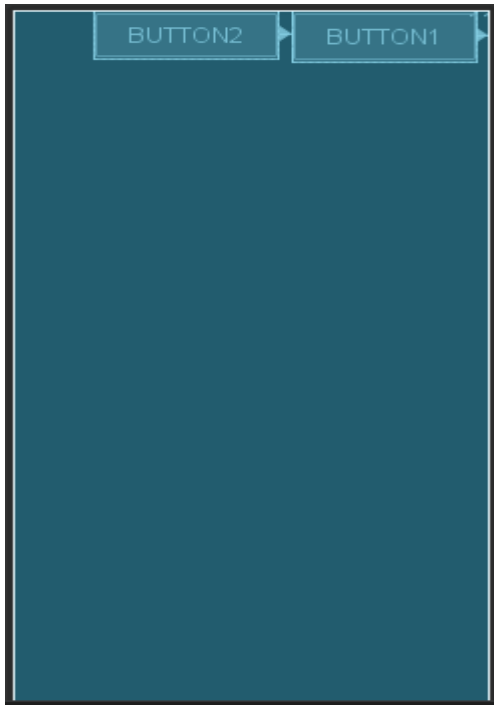


Layout attribute: XML File

- android:layout_width="fill_parent" : *Assigns layout width is equal with a width of the screen*
- android:layout_height="fill_parent“: *assigns layout height with a height of the screen*
- This means that the height and the width of the layout will be proportional to the screen
- Match_ parent: value can also be used instead of fill_parent in API Level 8 and higher
- Example :
  - android:layout_width="match_parent and android:layout_height="match_parent"
- wrap_content :
  - android:layout_height="wrap_content" and android:layout_width ="wrap_content"
- Sets the height of a container (i.e group view) to height of its contents (children).
- This means that its height and width will expand only far enough to contain its contents such as text or other child controls in it.

2. RelativeLayout

- Arranges view objects (its children) in relation to each other or to the parent.
- For Example,
  - If the first control is centered in the screen, other controls will be aligned relative to screen center
- Relative layout is often used to create forms.



2. RelativeLayout

- Example: XML file with Relative layout.

```xml
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">

    <Button
        android:id="@+id/txtUsername"
        android:layout_width="wrap_content"
        android:layout_height="57dp"
        android:layout_alignParentRight="true"
        android:layout_marginLeft="10dp"
        android:layout_marginRight="11dp"
        android:text="Button1"
        android:textSize="25sp" />

    <Button
        android:id="@+id/btnOk"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_toLeftOf="@+id/txtUsername"
        android:text="Button2"
        android:textSize="25sp" />
</RelativeLayout>
```

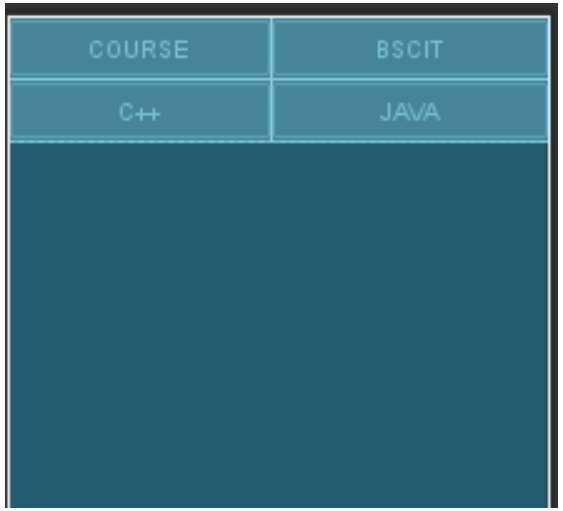
2. RelativeLayout : Example

- The children of the layout are identified by their unique identity values.
- For example:
  - Button android:id= "@+id/txtUsername“
  - Button android:id= "@+id/btnOk“
- In this example "@+id/ txtUsername " and"@+id/ btnOk " values are used to identify individual command buttons contained in the layout.
- The plus-symbol (+) means that this is a new resource name that must be created and added in application resources (in the R.java file).

3. TableLayout:

- **TableLayout** arranges controls (its children) in rows and columns, similar to an HTML table
- Each row has zero or more cells, each of which is defined by any kind of other View.
- So, the cells of a row may be composed of a variety of View objects, like ImageView or TextView objects.

- <TableLayout> element defines the beginning of a table layout.
- </TableLayout> element defines the end of a table layout.
- <TableRow> element specifies beginning of a table row.
- </TableRow> element defines end of a table row.

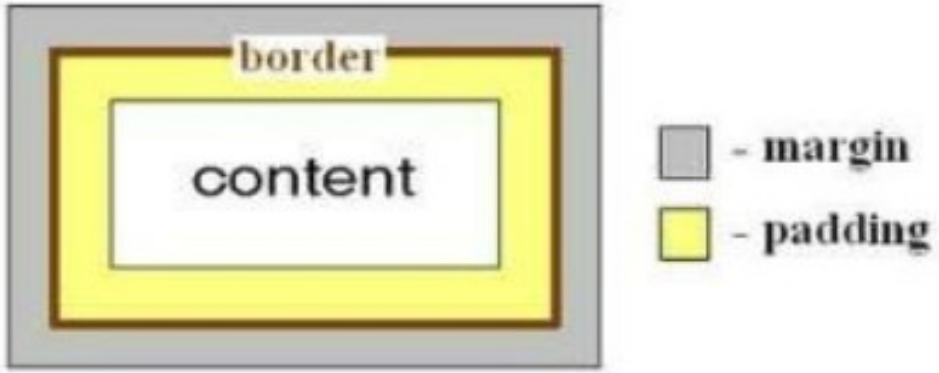


3. TableLayout: Example

```
<?xml version="1.0" encoding="utf-8"?>
<TableLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:stretchColumns="1">
    <TableRow
        android:layout_width="match_parent"
        android:layout_height="match_parent">
        <Button
            android:id="@+id/txtUsername1"
            android:layout_width="200dp"
            android:layout_height="match_parent"
            android:gravity="center"
            android:text="COURSE" />
        <Button
            android:id="@+id/btnOk"
            android:textStyle="bold"
            android:layout_width="wrap_content"
            android:layout_height="match_parent"
            android:text="BSCIT" />
    </TableRow>
</TableLayout>
```

3. TableLayout: Example

- android:padding="3dip"
- Sets the size of the space between the view border and its contents.
- In TextView padding is shifting a text inside it, but in a Layout view padding is shifting all layout elements.



4. FrameLayout:

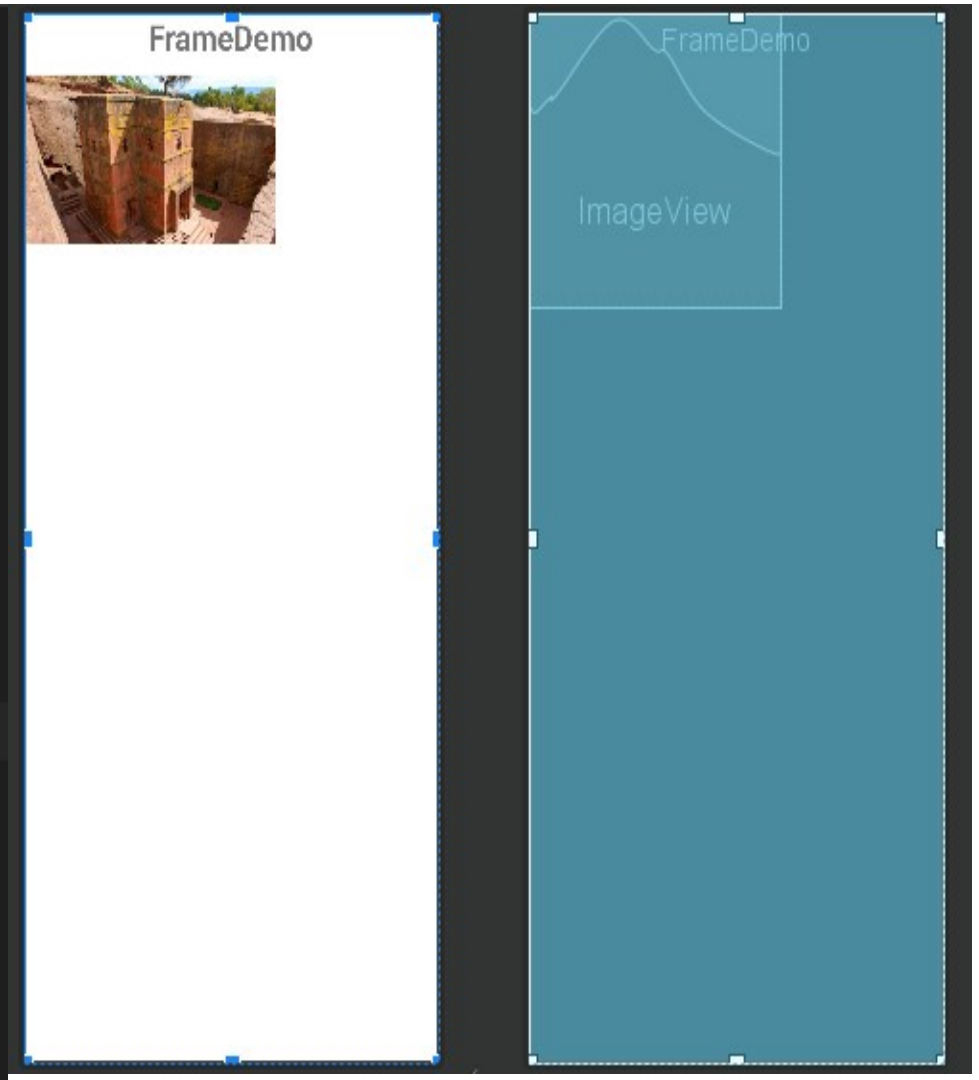
- FrameLayout creates a blank space on the screen that can be filled with a single object
- For example, a picture that you'll swap in and out.
- All its children(controls) start at the top left of the screen.
- you cannot specify a different location for a child view.
- Subsequent child views will simply be drawn over previous ones, partially or totally Obscuring them (unless the newer object is transparent).

4. FrameLayout:

- Generally, FrameLayout should be used to hold a single child view, because it can be difficult to organize child views in a way that's scalable to different screen sizes without the children overlapping each other.
- The FrameLayout is mostly used for tabbed views and image switchers.

4. FrameLayout: Example

```xml
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical">

    <ImageView
        android:id="@+id/txtUsername1"
        android:layout_width="250dp"
        android:layout_height="250dp"
        android:src="@drawable/lalibela"/>

    <TextView
        android:id="@+id/btnOk"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:text="FrameDemo"
        android:textAlignment="center"
        android:textSize="30sp"
        android:textStyle="bold" />
</FrameLayout>
```

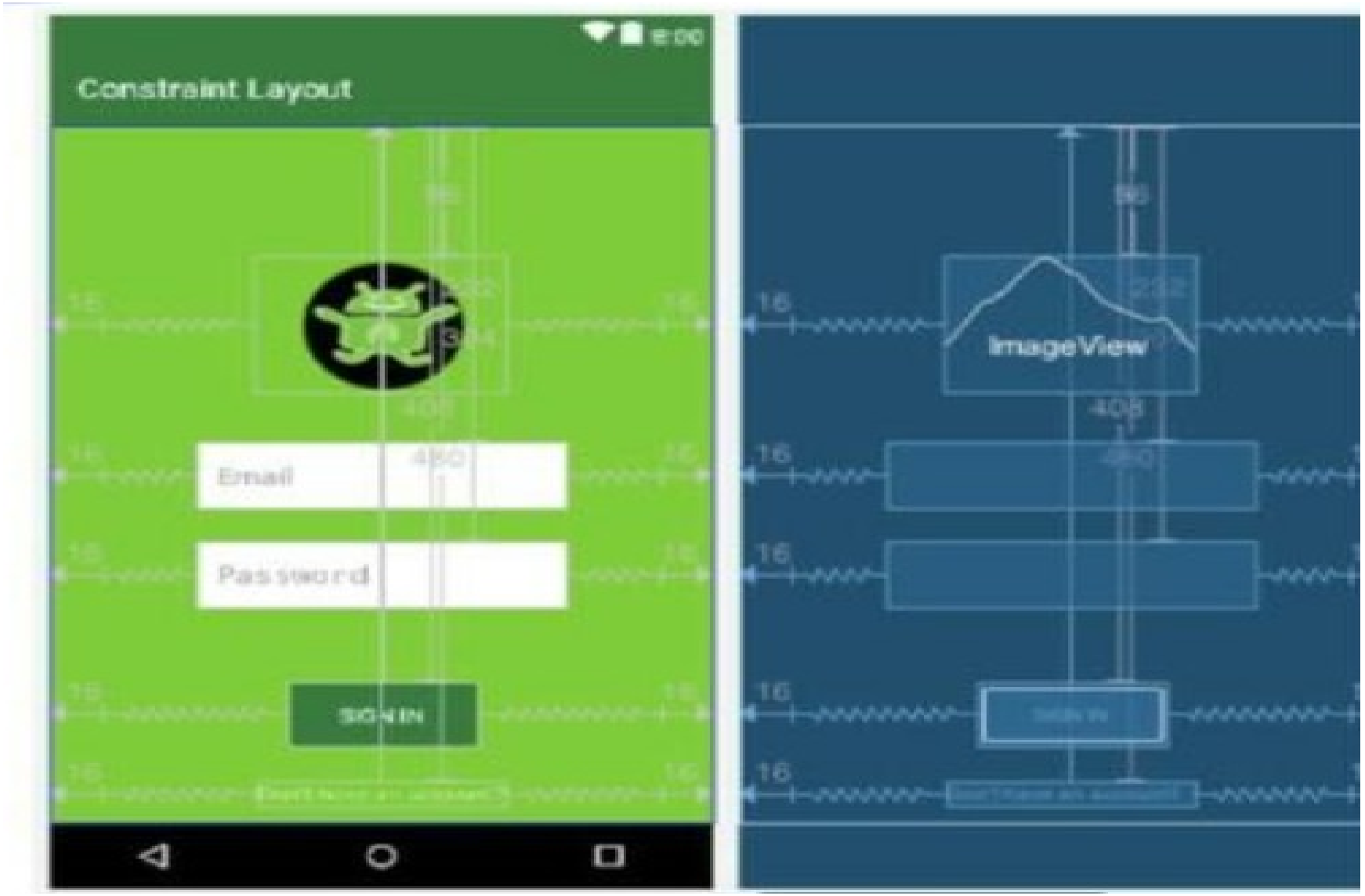


5. Constraint Layout

- **Constraint Layout** is a ViewGroup (i.e. a view that holds other views) which allows you to create large and complex layouts with a flat view hierarchy, and also allows you to position and size widgets in a very flexible way. It was created to help reduce the nesting of views and also improve the performance of layout files.

- **Constraint Layout** is very similar to **RelativeLayout** in such a way because, views are laid out according to relationships between sibling views and the parent layout yet it's a lot more flexible and works better with the Layout Editor of the Android Studio's.

5. Constraint Layout - diagram:



Constraint Layout

- Advantages Of Constraint Layout Over Other Layouts
  - One great advantage of the ***constraintlayout*** is that you can perform animations on your Constraint Layout views with very little code.
  - You can build your complete layout with simple drag-and-drop on the Android Studio design editor.
  - You can control what happens to a group of widgets through a single line of code.
  - Constraint Layout improve performance over other layout.

Loading Layout XML Resource

- Android compiles the XML layout code that is later loaded in java code by the following method:

```
Public void oncreate (Bundle savedinstancesstate)
{
    .....
    Setcontentview(R.layout.<layout_file_name>);
    ...
}
```

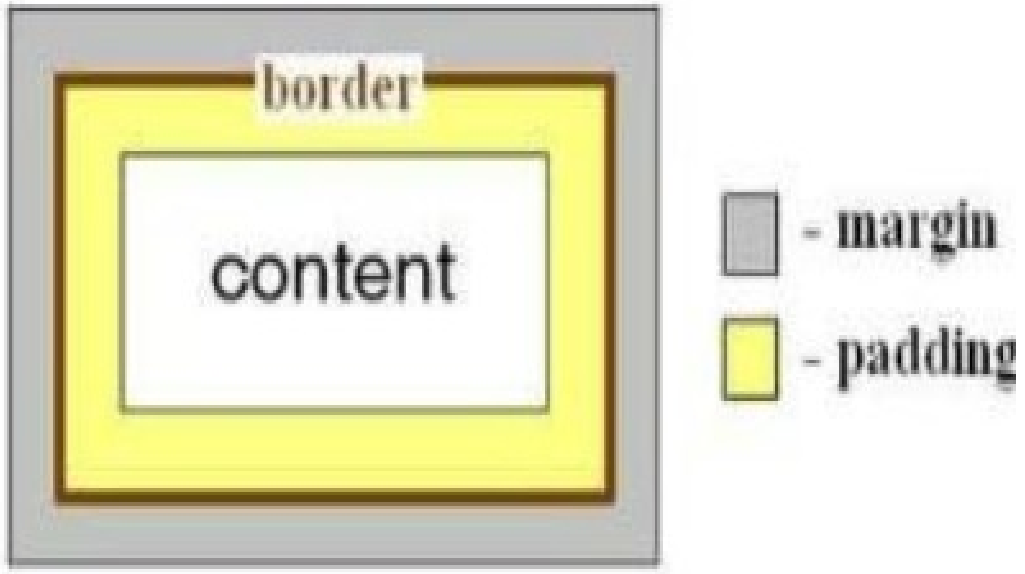
Loading Layout XML Resource

- A layout file is referenced using the following syntax:
  - R.layout.<layout_file_name>
- For instance, this example loads activity_main.xml file in the onCreate() method.

```
package com.application.hello_world;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class Registration extends AppCompatActivity
{
    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.registration_activity);
    }
}
```

Terminologies of Views and view groups

- Padding is the space inside the border, between the border and the actual view's content.
- Margins are the spaces outside the border, between the border and the other elements next to the view.
- Example:



Operations on views

- Common operations that can be performed on Views are:

1. Set properties by putting values of attributes in the XML layout files. E.g. Layout_width="fill parent", Layout_height="wrap content“
2. Set focus on view in response to user input by using requestFocus() method.
3. Set up listeners by putting listeners that will be notified when something interesting happens to the view control .For example, a Button notifies a listener to call clients when the button is clicked.
4. Set visibility by hiding or showing views using setVisibility(int) method

UI Components/ Widget

UI Controls

- Input controls are the interactive components in your app's user interface.
- Android provides a wide variety of controls to be used in UI, such as Buttons, Text Fields, Seek Bars, Check Box, Zoom Buttons, Toggle Buttons, and many more.

UI Elements

- A **View** is an object that draws something on the screen that the user can interact with and a **ViewGroup** is an object that holds other View (and ViewGroup) objects in order to define the layout of the user interface.

```
In activity_main.xml
<TextView
    android:id="@+id/lblWelcome"
    android:layout_width="wrap_content“
    android:layout_height="wrap_content"
    android:text=“Welcome" />
 <Button
    android:id="@+id/btnOK"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text=“OK" />
```

```
 In MainActivity.java: Access Resources by View by ID

TextView lblWelcome = (TextView) findViewById(R.id.lblWelcome);

Button btnOK= (TextView) findViewById(R.id.btnOK);
```

UI Components

1. **TextView**: Displays text to the user.
2. **EditText:** A predefined subclass of TextView that includes rich editing capabilities.
3. **AutoCompleteTextView:** A view that is similar to EditText, except that it shows a list of completion suggestions automatically while the user is typing.
4. **Button**: A push-button that can be pressed, or clicked, by the user to perform an action.
5. **ImageButton:** An AbsoluteLayout which enables you to specify the exact location of its children. This shows a button with an image (instead of text) that can be pressed or clicked by the user.
6. **CheckBox:** An on/off switch that can be toggled by the user. You should use check box when presenting users with a group of selectable options that are not mutually exclusive.
7. **ToggleButton:** An on/off button with a light indicator.
8. **RadioButton:** Has as two states: either checked or unchecked.
9. **RadioGroup:** Group together one or more RadioButtons.
10. **ProgressBar:** Provides visual feedback about some ongoing tasks, such as when you are performing a task in the background.
11. **Spinner**: A drop-down list that allows users to select one value from a set.
12. **TimePicker**: Enables users to select a time of the day, in either 24-hour mode or AM/PM mode.
13. **DatePicker**: Enables users to select a date of the day.

TextView

- Displays text to the user and optionally allows them to edit it.
- A complete text editor, however the basic class is configured to not allow editing.

Attribute	Description
android:id	This is the ID which uniquely identifies the control.
android:capitalize	If set, it has a textual input method and should automatically capitalize what the user types. • Don't automatically capitalize anything - 0 • Capitalize the first word of each sentence - 1 • Capitalize the first letter of every word - 2 • Capitalize every character - 3
android:cursorVisible	Makes the cursor visible (the default) or invisible. Default is false.
android:editable	If set to true, specifies that this TextView has an input method.
android:fontFamily	Font family (named by string) for the text.
android:gravity	Specifies how to align text by the view's x- and/or y-axis when the text is smaller than view.
android:hint	Hint text to display when the text is empty.
android:inputType	The type of data being placed in a text field. Phone, Date, Time, Number, Password etc.
android:maxHeight	Makes the TextView be at most this many pixels tall.
android:maxWidth	Makes the TextView be at most this many pixels wide.

...Cont'd

Attribute	Description
android:minHeight	Makes the TextView be at least this many pixels tall.
android:minWidth	Makes the TextView be at least this many pixels wide.
android:password	Whether the characters of the field are displayed as password dots instead of themselves. Possible value either "true" or "false".
android:phoneNumber	If set, it has a phone number input method. Possible value either "true" or "false".
android:text	Text to display.
android:textAllCaps	Present the text in ALL CAPS. Possible value either "true" or "false".
android:textColor	Text color. May be a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".
android:textColorHighlight	Color of the text selection highlight.
android:textColorHint	Color of the hint text. May be a color value, in the form of "#rgb", "#argb", "#rrggbb", or "#aarrggbb".
android:textIsSelectable	Indicates that content of a non-editable text can be selected. Possible value either "true" or "false".

…Cont'd

Attribute	Description
`android:textSize`	Size of the text. Recommended dimension type for text is "sp" for scaled-pixels (example: 15sp).
`android:textStyle`	Style (bold, italic, bolditalic) for the text. You can use or more of the following values separated by '|'. •normal - 0 •bold - 1 •italic - 2
`android:typeface`	Typeface (normal, sans, serif, monospace) for the text. You can use or more of the following values separated by '|'. •normal - 0 •sans - 1 •serif - 2 •monospace - 3

❑ TextView Example

```
<TextView
    android:id="@+id/txtHello"
    android:layout_width="300dp"
    android:layout_height="200dp“
    android:capitalize="characters"
    android:text="Hello World"
    android:textColor="@android:color/holo_blue_dark"
    android:textColorHighlight="@android:color/primary_text_dark"
    android:layout_centerVertical="true"
    android:layout_alignParentEnd="true"
    android:textSize="50dp"/>
```

Lab TextView Exercise

- Try above example with different attributes of TextView in Layout XML file as well at programming time to have different look and feel of the TextView.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple TextView controls in one activity.

EditText

- An overlay over TextView that configures itself to be editable.
- The predefined subclass of TextView that includes rich editing capabilities.
- Attributes Inherited from `android.widget.TextView` **Class** –

Attribute	Description
android:autoText	If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.
android:drawableBottom	This is the drawable to be drawn below the text.
android:drawableRight	This is the drawable to be drawn to the right of the text.
android:editable	If set, specifies that this TextView has an input method.
android:text	This is the Text to display.

…Cont'd

- Inherited from `android.view.View` Class –

Attribute	Description
`android:background`	This is a drawable to use as the background.
`android:contentDescription`	This defines text that briefly describes content of the view.
`android:id`	This supplies an identifier name for this view.
`android:onClick`	This is the name of the method in this View's context to invoke when the view is clicked.
`android:visibility`	This controls the initial visibility of the view.

❑ EditText Example: activity_main.xml

```
<EditText
   android:id="@+id/txtFirstName"
   android:layout_width="fill_parent"
   android:layout_height="wrap_content"
   android:ems="10"
   android:text="@string/enter_text"
   android:inputType="text" />

<Button
   android:id="@+id/btnOK"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="@string/show_the_text" />
```

…Cont'd

```
public class MainActivity extends Activity {
    EditText inputFirstName;
    Button btnOK;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        inputFirstName = (EditText) findViewById(R.id.txtFirstName);
        btnOK = (Button) findViewById(R.id.btnOK);
        btn.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {
            String firstName = inputFirstName.getText().toString();
            Toast msg = Toast.makeText(getBaseContext(), firstName,
            Toast.LENGTH_LONG); msg.show();
          }
       });
    }
}
```

EditText Lab Exercise:

- Try above example with different attributes of EditText in Layout XML file as well at programming time to have different look and feel of the EditText.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple EditText controls in one activity.

AutoCompleteTextView

- A view that is similar to EditText, except that it shows a list of completion suggestions automatically while the user is typing.

Attribute	Description
android:completionHint	This defines the hint displayed in the drop down menu.
android:completionHintView	This defines the hint view displayed in the drop down menu.
android:completionThreshold	This defines the number of characters that the user must type before completion suggestions are displayed in a drop down menu.
android:dropDownAnchor	This is the View to anchor the auto-complete dropdown to.
android:dropDownHeight	This specifies the basic height of the dropdown.
android:dropDownHorizontalOffset	The amount of pixels by which the drop-down should be offset horizontally.
android:dropDownSelector	This is the selector in a drop down list.
android:dropDownVerticalOffset	The amount of pixels by which the drop down should be offset vertically.
android:dropDownWidth	This specifies the basic width of the dropdown.
android:popupBackground	This sets the background.

❑ AutoCompleteTextView Example: MainActivity.java

```
public class MainActivity extends Activity {

    AutoCompleteTextView autocomplete;

    String[] arr = { "Paris, France", "PA, United States","Parana, Brazil", "Padua,
    Italy", "Pasadena, CA, United States"};

    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        autocomplete = (AutoCompleteTextView) findViewById(R.id.autoCompleteTextView1);

        ArrayAdapter<String> adapter = new ArrayAdapter<String>
        (this,android.R.layout.select_dialog_item, arr);

        autocomplete.setThreshold(2);
        autocomplete.setAdapter(adapter);
    }
}
```

❑ AutoCompleteTextView Example: activity_main.xml

```
<AutoCompleteTextView
    android:id="@+id/autoCompleteTextView1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:ems="10" />
```

Lab Exercise

- Try above example with different attributes of AutoCompleteTextView in Layout XML file as well at programming time to have different look and feel of the AutoCompleteTextView.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple AutoCompleteTextView controls in one activity.

Button

- Push-button which can be pressed, or clicked, by the user to perform an action..
- Attributes Inherited from `android.widget.TextView` **Class** –

Attribute	Description
`android:autoText`	If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.
`android:drawableBottom`	This is the drawable to be drawn below the text.
`android:drawableRight`	This is the drawable to be drawn to the right of the text.
`android:editable`	If set, specifies that this TextView has an input method.
`android:text`	This is the Text to display.

…Cont'd

- Inherited from `android.view.View` Class –

Attribute	Description
`android:background`	This is a drawable to use as the background.
`android:contentDescription`	This defines text that briefly describes content of the view.
`android:id`	This supplies an identifier name for this view.
`android:onClick`	This is the name of the method in this View's context to invoke when the view is clicked.
`android:visibility`	This controls the initial visibility of the view.

❑ Button Example: MainActivity.java

```
public class MainActivity extends Activity {
    EditText inputFirstName;
    Button btnOK;

    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        inputFirstName = (EditText) findViewById(R.id.txtFirstName);
        btnOK = (Button) findViewById(R.id.btnOK);
        btn.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {
            String firstName = inputFirstName.getText().toString();
            Toast msg = Toast.makeText(getBaseContext(), firstName, Toast.LENGTH_LONG);
            msg.show();
          }
      });
    }
}
```

❑ Button Example: activity_main.xml

```
<EditText
   android:id="@+id/txtFirstName"
   android:layout_width="fill_parent"
   android:layout_height="wrap_content"
   android:ems="10"
   android:text="@string/enter_text"
   android:inputType="text" />

<Button
   android:id="@+id/btnOK"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="@string/show_the_text" />
```

Lab Button Exercise:

- Try above example with different attributes of Button in Layout XML file as well at programming time to have different look and feel of the Button.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple Button controls in one activity.

ImageButton

- AbsoluteLayout which enables you to specify the exact location of its children.
- Shows a button with an image (instead of text) that can be pressed or clicked by user.

ImageButton

- Attributes Inherited from `android.widget.ImageView` **Class** –

Attribute	Description
`android:adjustViewBounds`	Set this to true if you want the ImageView to adjust its bounds to preserve the aspect ratio of its drawable.
`android:baseline`	This is the offset of the baseline within this view.
`android:baselineAlignBottom`	If true, the image view will be baseline aligned with based on its bottom edge.
`android:cropToPadding`	If true, the image will be cropped to fit within its padding.
`android:src`	This sets a drawable as the content of this ImageView.

…Cont'd

- Inherited from `android.view.View` Class –

Attribute	Description
`android:background`	This is a drawable to use as the background.
`android:contentDescription`	This defines text that briefly describes content of the view.
`android:id`	This supplies an identifier name for this view.
`android:onClick`	This is the name of the method in this View's context to invoke when the view is clicked.
`android:visibility`	This controls the initial visibility of the view.

❑ ImageButton Example: MainActivity.java

…Cont'd

```
public class MainActivity extends Activity {

    ImageButton btnOK;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        btnOK = (ImageButton) findViewById(R.id.btnImageOK);

        btnOK.setOnClickListener(new OnClickListener() {
            public void onClick(View v) {

            Toast msg = Toast.makeText(getBaseContext(), "MESSAGE HERE", Toast.LENGTH_LONG);
            msg.show();
          }
      });
    }
}
```

❑ ImageButton Example: activity_main.xml

```
<ImageButton
    android:id="@+id/btnImageOK"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/show_the_text" />
```

ImageButton Exercise

- Try above example with different attributes of ImageButton in Layout XML file as well at programming time to have different look and feel of the ImageButton.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple ImageButton controls in one activity.

CheckBox

- A CheckBox is an on/off switch that can be toggled by the user.
- Use check-boxes when presenting users with a group of selectable options that are not mutually exclusive.
- Attributes Inherited from `android.widget.TextView` **Class** –

Attribute	Description
`android:autoText`	If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.
`android:drawableBottom`	This is the drawable to be drawn below the text.
`android:drawableRight`	This is the drawable to be drawn to the right of the text.
`android:editable`	If set, specifies that this TextView has an input method.
`android:text`	This is the Text to display.

…Cont'd

- Inherited from `android.view.View` Class –

Attribute	Description
`android:background`	This is a drawable to use as the background.
`android:contentDescription`	This defines text that briefly describes content of the view.
`android:id`	This supplies an identifier name for this view.
`android:onClick`	This is the name of the method in this View's context to invoke when the view is clicked.
`android:visibility`	This controls the initial visibility of the view.

CheckBox Example: MainActivity.java

...Cont'd

```
public class MainActivity extends Activity
 {
        CheckBox ch1,ch2;
        Button b1,b2;

        @Override
        protected void onCreate(Bundle savedInstanceState) {
                super.onCreate(savedInstanceState);
                setContentView(R.layout.activity_main);

                ch1 = (CheckBox)findViewById(R.id.chkCheckBox1);
                ch2 = (CheckBox)findViewById(R.id.chkCheckBox2);
                b1 = (Button)findViewById(R.id.btnButton1);
                b2 = (Button)findViewById(R.id.btnButton2);

                b2.setOnClickListener(new OnClickListener() {
                        public void onClick(View v) {
                         finish()
                        }
                  });
                b2.setOnClickListener(new OnClickListener() {
                        public void onClick(View v) {
                                StringBuffer result = new StringBuffer();
                                result.append("Thanks : ").append(ch1.isChecked());
                                result.append("\nThanks: ").append(ch2.isChecked());
                                Toast.makeText(MainActivity.this, result.toString(), Toast.LENGTH_LONG).show();
                        }
                  });
        }
}
```

CheckBox Example: activity_main.xml

```
<CheckBox
   android:id="@+id/chkCheckBox1"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="Do you like Programming?"
   android:layout_centerHorizontal="true" />

<CheckBox
   android:id="@+id/chkCheckBox2"
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="Do you like Android?"
   android:checked="false”/>
```

…Cont'd

Lab CheckBox Exercise:

- Try above example with different attributes of CheckBox in Layout XML file as well at programming time to have different look and feel of the CheckBox.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple CheckBox controls in one activity.

ToggleButton

- Displays checked/unchecked states as a button. It is basically an on/off button with a light indicator.

Attribute	Description
`android:disabledAlpha`	This is the alpha to apply to the indicator when disabled.
`android:textOff`	This is the text for the button when it is not checked.
`android:textOn`	This is the text for the button when it is checked.

…Cont'd

- Inherited from `android.view.TextView` Class –

Attribute	Description
`android:autoText`	If set, specifies that this TextView has a textual input method and automatically corrects some common spelling errors.
`android:drawableBottom`	This is the drawable to be drawn below the text.
`android:drawableRight`	This is the drawable to be drawn to the right of the text.
`android:editable`	If set, specifies that this TextView has an input method.
`android:text`	This is the Text to display.

…Cont'd

- Inherited from `android.view.View` Class –

Attribute	Description
`android:background`	This is a drawable to use as the background.
`android:contentDescription`	This defines text that briefly describes content of the view.
`android:id`	This supplies an identifier name for this view.
`android:onClick`	This is the name of the method in this View's context to invoke when the view is clicked.
`android:visibility`	This controls the initial visibility of the view.

ToggleButton Example: MainActivity.java

```
public class MainActivity extends ActionBarActivity {
    ToggleButton tg1,tg2;
    Button b1;
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        tg1=(ToggleButton)findViewById(R.id.toggleButton);
        tg2=(ToggleButton)findViewById(R.id.toggleButton2);
        b1=(Button)findViewById(R.id.button2);
        b1.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            StringBuffer result = new StringBuffer();
            result.append("You have clicked first ON Button-:) ").append(tg1.getText());
            result.append("You have clicked Second ON Button -:)").append(tg2.getText());
            Toast.makeText(MainActivity.this, result.toString(),Toast.LENGTH_SHORT).show();
          }
        });
    }
}
```

❑ ToggleButton Example: activity_main.xml

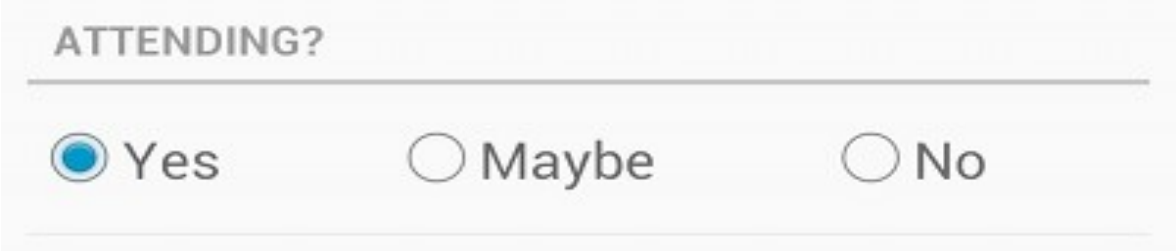
```
<ToggleButton
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="On"
   android:id="@+id/toggleButton"
   android:checked="true"/>

<ToggleButton
   android:layout_width="wrap_content"
   android:layout_height="wrap_content"
   android:text="Off"
   android:id="@+id/toggleButton2"
   android:checked="true"/>
```

ToggleButton Exercise:

- Try above example with different attributes of ToggleButton in Layout XML file as well at programming time to have different look and feel of the ToggleButton.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple ToggleButton controls in one activity.

RadioButton

- Has two states: either checked or unchecked.
- Allows the user to select one option from a set.



…Cont'd

❑ RadioButton Example: MainActivity.java

```
public class MainActivity extends ActionBarActivity {
    RadioGroup rg1;
    RadioButton rb1;
    Button b1;
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        addListenerRadioButton();
    }
    private void addListenerRadioButton() {
        rg1 = (RadioGroup) findViewById(R.id.radioGroup);
        b1 = (Button) findViewById(R.id.button2);
        b1.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) {
            int selected=rg1.getCheckedRadioButtonId();
            rb1=(RadioButton)findViewById(selected);
            Toast.makeText(MainActivity.this,rb1.getText(),Toast.LENGTH_LONG).show();
            }
        });
    }
}
```

❑ RadioButton Example: activity_main.xml

…Cont'd

```
<RadioGroup
    android:id="@+id/radioGroup"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:layout_alignLeft="@+id/textView2"
    android:layout_alignStart="@+id/textView2">
    <RadioButton
        android:layout_width="142dp"
        android:layout_height="wrap_content"
        android:text="JAVA"
        android:id="@+id/radioButton"
        android:textSize="25dp"
        android:textColor="@android:color/
        holo_red_light"
        android:checked="false"
        android:layout_gravity="center_horizon
        tal" />
    <RadioButton
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="ANDROID"
        android:id="@+id/radioButton2"
        android:layout_gravity="center_horizontal"
        android:checked="false"
        android:textColor="@android:color/holo_red_dark"
        android:textSize="25dp" />
    <RadioButton
        android:layout_width="136dp"
        android:layout_height="wrap_content"
        android:text="HTML"
        android:id="@+id/radioButton3"
        android:layout_gravity="center_horizontal"
        android:checked="false"
        android:textSize="25dp"
        android:textColor="@android:color/holo_red_dark" />
</RadioGroup>
```

Lab RadioButton Exercise:

- Try above example with different attributes of **RadioButton** in Layout XML file as well at programming time to have different look and feel of the **RadioButton**.
- Try to make it editable, change to font color, font family, width, textSize etc and see the result.
- Try above example with multiple **RadioButton** controls in one activity.

ProgressBar

- Shows progress of a task.
- For example, while uploading or downloading (show the progress of download or upload to user).
- In android, a class called ProgressDialog allows to create progress bar.
- To do this, instantiate an object of this class. The syntax is:

```
ProgressDialog progress = new ProgressDialog(this);
```

- Set some properties of this dialog such as, its style, its text etc.

```
progress.setMessage("Downloading Music:) ");
progress.setProgressStyle(ProgressDialog.STYLE_HORIZONTAL);
progress.setIndeterminate(true);
```

…Cont'd

- Methods provided by `ProgressDialog Class` –

Attribute	Description
`getMax()`	This method returns the maximum value of the progress.
`incrementProgressBy(int diff)`	This method increments the progress bar by the difference of value passed as a parameter.
`setIndeterminate(boolean indeterminate)`	This method sets the progress indicator as determinate or indeterminate.
`setMax(int max)`	This method sets the maximum value of the progress dialog.
`setProgress(int value)`	This method is used to update the progress dialog with some specific value.
`show(Context context, CharSequence title, CharSequence message)`	This is a static method, used to display progress dialog.

❑ ProgressBar Example: MainActivity.java

…Cont'd

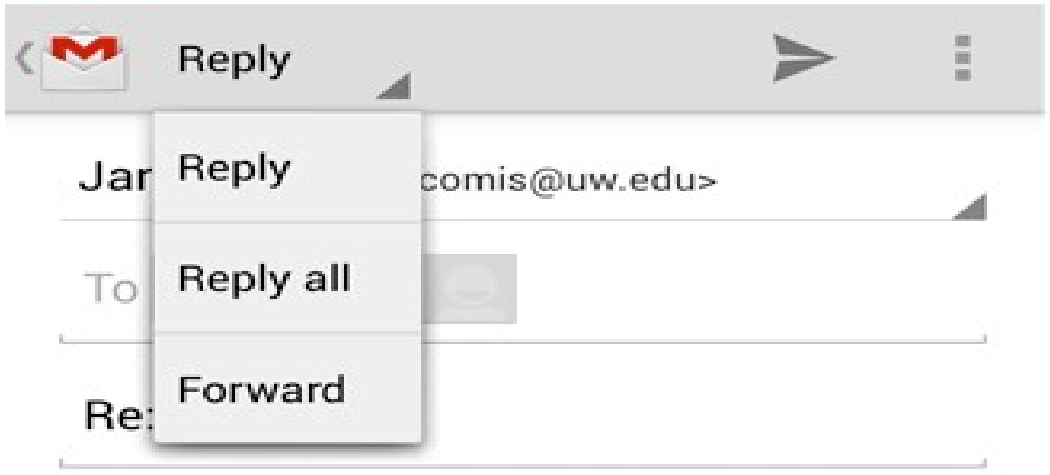
```
public class MainActivity extends ActionBarActivity {


    Button btnDownload;
    private ProgressDialog progress;
    protected void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        btnDownload = (Button) findViewById(R.id.btnDownload);
    }

    public void download(View view){
        progress=new ProgressDialog(this);
        progress.setMessage("Downloading Music");
        progress.setProgressStyle(ProgressDialog.STYLE_HORIZONTAL);
        progress.setIndeterminate(true);
        progress.setProgress(0);
        progress.show();
        final int totalProgressTime = 100;
        final Thread t = new Thread() {
            @Override
            public void run() {
                int jumpTime = 0;
                while(jumpTime < totalProgressTime) {
                    try {
                        sleep(200);
                        jumpTime += 5;
                        progress.setProgress(jumpTime);
                    }
                    catch (InterruptedException e) {
                    // TODO Auto-generated catch block
                    e.printStackTrace();
                    }
                }
            }
        };
        t.start();
    }
}
```

❑ ProgressBar Example: activity_main.xml

```
<Button
    android:id="@+id/btnDownload"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content“
    android:text="Download"
    android:onClick="download"
    android:layout_marginLeft="125dp"
    android:layout_marginStart="125dp“
    android:layout_centerVertical="true" />
```

Spinner

- Spinner allows you to select an item from a drop down menu.
- For example:
  - Gmail application has drop down menu as shown below, select an item you want from a drop down menu.



❑ Spinner Example: MainActivity.java

```
class AndroidSpinnerExampleActivity extends Activity implements OnItemSelectedListener{

    @Override
    public void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.main);

        // Spinner element
        Spinner spinner = (Spinner) findViewById(R.id.spinner);

        // Spinner click listener
        spinner.setOnItemSelectedListener(this);

        // Spinner Drop down elements
        List<String> categories = new ArrayList<String>();
        categories.add("Automobile");
        categories.add("Business Services");
        categories.add("Computers");
        categories.add("Education");
        categories.add("Personal");
        categories.add("Travel");
```

❑ Spinner Example: MainActivity.java

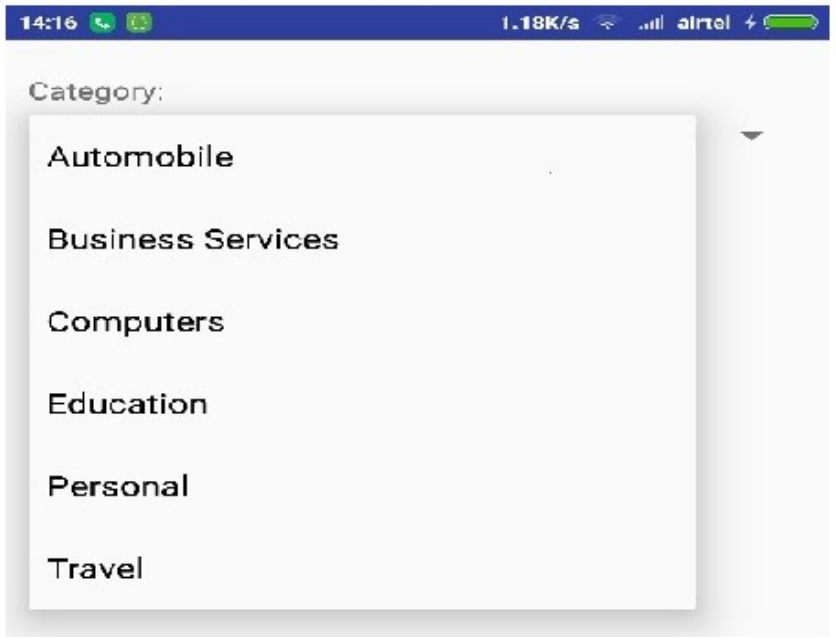
```
            // Creating adapter for spinner
        ArrayAdapter<String> dataAdapter = new ArrayAdapter<String>(this,
        android.R.layout.simple_spinner_item, categories);
            // Drop down layout style - list view with radio button
        dataAdapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
            // attaching data adapter to spinner
        spinner.setAdapter(dataAdapter);
    }

    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        // On selecting a spinner item
        String item = parent.getItemAtPosition(position).toString();
        // Showing selected spinner item
        Toast.makeText(parent.getContext(), "Selected: " + item, Toast.LENGTH_LONG).show();
    }

    public void onNothingSelected(AdapterView<?> arg0) {
    // TODO Auto-generated method stub
    }
}
```

- Spinner Example: activity_main.xml

```
<Spinner
   android:id="@+id/spinner"
   android:layout_width="fill_parent"
   android:layout_height="wrap_content"
   android:prompt="@string/spinner_title"/>
```

- Spinner Example: Modify the res/values/string.xml to the following.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">AndroidSpinnerExample</string>
</resources>
```



TimePicker

- Allows to select the time of day in either 24 hour or AM/PM mode.
- The time consists of hours, minutes and clock format.
- Android provides this functionality through TimePicker class.

TimePicker Example: activity_main.xml

```
<TimePicker
android:id="@+id/timePicker1"
android:layout_width="wrap_content"
android:layout_height="wrap_content" />
```

…Cont'd

- Methods in the API that gives more control over TimePicker Component.

Attribute	Description
`is24HourView()`	This method returns true if this is in 24 hour view else false
`isEnabled()`	This method returns the enabled status for this view
`setCurrentHour(Integer currentHour)`	This method sets the current hour
`setCurrentMinute(Integer currentMinute)`	This method sets the current minute
`setEnabled(boolean enabled)`	This method set the enabled state of this view
`setIs24HourView(Boolean is24HourView)`	This method set whether in 24 hour or AM/PM mode
`setOnTimeChangedListener(TimePicker.OnTimeChangedListener onTimeChangedListener)`	This method Set the callback that indicates the time has been adjusted by the user

❑ TimePicker Example: activity_main.xml

```
<TextView
     android:id="@+id/textView2"
     android:layout_width="wrap_content“
      android:layout_height="wrap_content"
     android:layout_alignParentTop="true"
     android:layout_centerHorizontal="true"
     android:text="@string/time_pick"
     android:textAppearance="?android:attr/textAppearanceMedium" />
<Button
     android:id="@+id/set_button"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:layout_alignParentBottom="true"
     android:layout_centerHorizontal="true"
     android:layout_marginBottom="180dp"
     android:onClick="setTime"
     android:text="@string/time_save" />
<TimePicker
     android:id="@+id/timePicker1"
     android:layout_width="wrap_content"
     android:layout_height="wrap_content"
     android:layout_above="@+id/set_button"
     android:layout_centerHorizontal="true"
     android:layout_marginBottom="24dp" />
```

❑ TimePicker Example: activity_main.xml

```
<TextView
    android:id="@+id/textView3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignLeft="@+id/timePicker1"
    android:layout_alignTop="@+id/set_button"
    android:layout_marginTop="67dp"
    android:text="@string/time_current"
    android:textAppearance="?android:attr/textAppearanceMedium" />

<TextView
    android:id="@+id/textView1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_below="@+id/textView3"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="50dp"
    android:text="@string/time_selected"
    android:textAppearance="?android:attr/textAppearanceMedium" />
```

❑ TimePicker Example: MainActivity.java

```
public class MainActivity extends Activity {
private TimePicker timePicker1;
private TextView time;
private Calendar calendar;
private String format = "";
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    timePicker1 = (TimePicker) findViewById(R.id.timePicker1);
    time = (TextView) findViewById(R.id.textView1);
    calendar = Calendar.getInstance();
    int hour = calendar.get(Calendar.HOUR_OF_DAY);
    int min = calendar.get(Calendar.MINUTE); showTime(hour, min);
 }
```

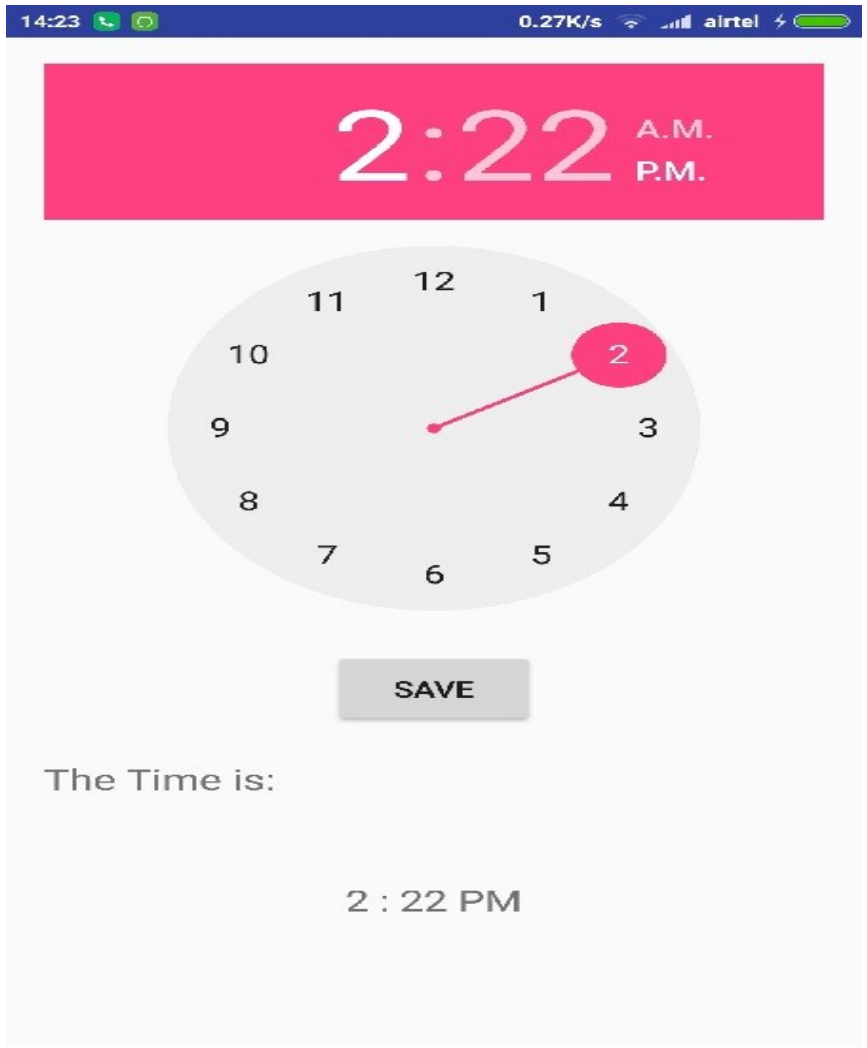
TimePicker Example: MainActivity.java

```
    public void setTime(View view) {
        int hour = timePicker1.getCurrentHour();
        int min = timePicker1.getCurrentMinute();
        showTime(hour, min);
    }
    public void showTime(int hour, int min) {
        if (hour == 0) {
            hour += 12;
            format = "AM";
        }
        else if (hour == 12) {
            format = "PM";
         } else if (hour > 12) {
            hour -= 12;
            format = "PM";
        }  else {
            format = "AM";
        }
        time.setText(new StringBuilder().append(hour).append(" : ").append(min).append(" ").
        append(format));
    }
}
```

TimePicker Example: Following is the content of the res/values/string.xml.

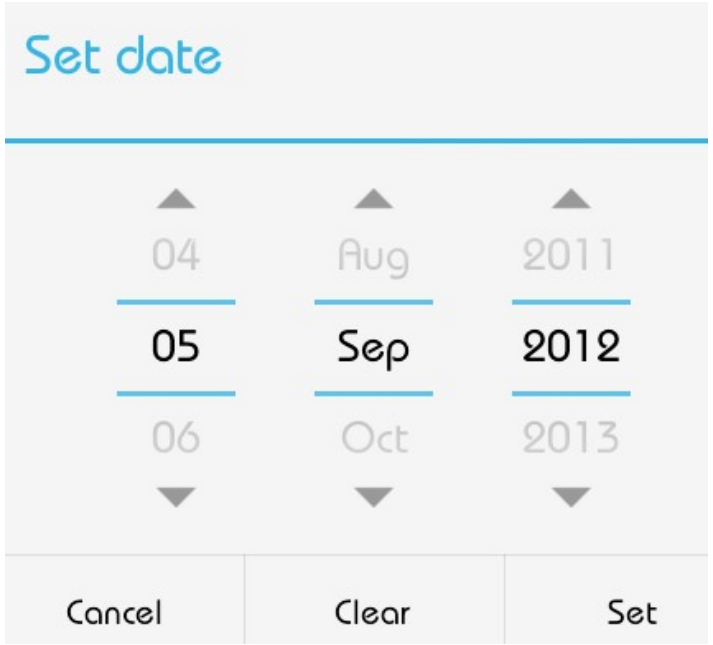
```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="app_name">TimePicker</string>
    <string name="action_settings">Settings</string>
    <string name="Time_picker_example">TimePicker Example</string>
    <string name="Time_pick">Pick the time and press save button</string>
    <string name="Time_save">Save</string>
    <string name="Time_selected">Cancel</string>
    <string name="Time_current">The Time is:</string>
</resources>
```

TimePicker Example:



DatePicker

- Allows to select the date consisting of day, month and year in your custom user interface.
- Android provides DatePicker and DatePickerDialog components.
- DatePickerDialog is a simple dialog containing DatePicker



- **DatePicker** object is also passed into this function.
- Use the following methods of the **DatePicker** to perform further operation.

Attribute	Description
getDayOfMonth()	Gets the selected day of month
getMonth()	Gets the selected month
getYear()	Gets the selected year
setMaxDate(long maxDate)	Sets the maximal date supported by this DatePicker in milliseconds since January 1, 1970 00:00:00 in getDefault() time zone
setMinDate(long minDate)	Sets the minimal date supported by this NumberPicker in milliseconds since January 1, 1970 00:00:00 in getDefault() time zone
setSpinnersShown(boolean shown)	Sets whether the spinners are shown
updateDate(int year, int month, int dayOfMonth)	Updates the current date
getCalendarView()	Returns calendar view
getFirstDayOfWeek()	Returns first day of the week

❑ DatePicker Example: activity_main.xml

```
<Button
    android:id="@+id/button1“
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentTop="true"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="70dp"
    android:onClick="setDate"
    android:text="@string/date_button_set" />

<TextView
    android:id="@+id/textView1"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignParentTop="true"
    android:layout_centerHorizontal="true"
    android:layout_marginTop="24dp"
    android:text="@string/date_label_set"
    android:textAppearance="?android:attr/textAppearanceMedium" />
```

DatePicker Example: activity_main.xml

```
<TextView
    android:id="@+id/textView2"
    android:layout_width="wrap_content“
    android:layout_height="wrap_content"
    android:layout_below="@+id/button1"
    android:layout_marginTop="66dp"
    android:layout_toLeftOf="@+id/button1"
    android:text="@string/date_view_set"
    android:textAppearance="?android:attr/textAppearanceMedium" />

<TextView
    android:id="@+id/textView3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_alignRight="@+id/button1"
    android:layout_below="@+id/textView2"
    android:layout_marginTop="72dp"
    android:text="@string/date_selected"
    android:textAppearance="?android:attr/textAppearanceMedium”/>
```

❑ DatePicker Example: Following is the content of the res/values/string.xml.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <resources> <string name="app_name">DatePicker</string>
    <string name="action_settings">Settings</string>
    <string name="hello_world">Hello world!</string>
    <string name="date_label_set">Press the button to set the date</string>
    <string name="date_button_set">Set Date</string>
    <string name="date_view_set">The Date is: </string>
    <string name="date_selected"></string>
</resources>
```

❑ DatePicker Example: MainActivity.java

…Cont'd

```
public class MainActivity extends Activity {
    private DatePicker datePicker;
    private Calendar calendar;
    private TextView dateView;
    private int year, month, day;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        dateView = (TextView) findViewById(R.id.textView3);
            calendar = Calendar.getInstance();
            year = calendar.get(Calendar.YEAR);
            month = calendar.get(Calendar.MONTH);
            day = calendar.get(Calendar.DAY_OF_MONTH);
        showDate(year, month+1, day);
    }
    @SuppressWarnings("deprecation")
    public void setDate(View view)
    {
        showDialog(999);
        Toast.makeText(getApplicationContext(), "ca", Toast.LENGTH_SHORT) .show();
    }
```

❑ DatePicker Example: MainActivity.java

…Cont'd

```
    @Override
    protected Dialog onCreateDialog(int id) {
        // TODO Auto-generated method stub
        if (id == 999) {
            return new DatePickerDialog(this, myDateListener, year, month, day);
        }
        return null;
    }
    private DatePickerDialog.OnDateSetListener myDateListener = new
    DatePickerDialog.OnDateSetListener() {
    @Override
    public void onDateSet(DatePicker arg0, int arg1, int arg2, int arg3) {
        // TODO Auto-generated method stub // arg1 = year // arg2 = month // arg3 = day
        showDate(arg1, arg2+1, arg3);
      }
    };
    private void showDate(int year, int month, int day) {
        dateView.setText(new StringBuilder().append(day).append("/") .append(month).
        append("/").append(year));
    }
}
```

❑ DatePicker Example:

Notification, Menus and Dialogs

Notification, Menus and Dialogs

- Intents with Parameter
- Notification with Status Bar and Toast
- Localization
- Context/Option Menu
- Alert Dialog

Notification, Menus and Dialogs

- **Notification**
  - Message displayed outside of the app's UI to provide user with information or updates.
  - Displayed in notification bar, on the lock screen, or as a heads-up
- **Menu**
  - List of options or actions that can be accessed by the user in various ways.
  - As options menu (accessed via the menu button), a context menu (displayed when the user long-presses on an element), or an action bar (displayed at the top of the screen).
- **Dialog**
  - Floating window used to display important information or prompt user for a decision.
  - Used to display error messages, confirmations, or input prompts.
  - Alternative to an Activity, allowing the user to interact with the dialog while remaining in current activity.

Intents with Parameter

- Intent is a message object used to request an action from another app component, such as starting an activity or service.
- An intent with a parameter can be used to pass data between different components within an app or between different apps.
- For example, consider an app that allows users to view and edit their contacts.
- The app might have an activity for displaying a list of contacts and another activity for displaying the details of a single contact.
- When a user clicks on a contact in the list, the app can use an intent with a parameter to start the "view contact" activity and pass the ID of the selected contact as a parameter.
- To create an intent with a parameter, you would use the Intent class in Android and set the data and/or extras properties to include the parameter.

…Cont'd

- To create an intent with a parameter, you would use the Intent class in Android and set the data and/or extras properties to include the parameter.

```
Intent intent = new Intent(this, ViewContactActivity.class);
intent.putExtra("contact_id", 1);
startActivity(intent);
```

- In the above example, we are creating an intent to start an activity called `ViewContactActivity`.
- We are passing an extra parameter called "contact_id" with a value of 1 as an extra to the Intent.
- In the `ViewContactActivity` we can retrieve this extra using following code snippet

```
Intent contactId = getIntent().getExtra("contact_id", 1);
```

…Cont'd

- Example: To pass the parameters, create new intent and put a parameter map:

```
Intent myIntent = new Intent(this, NewActivityClassName.class);
myIntent.putExtra("firstKeyName", "FirstKeyValue");
myIntent.putExtra("secondKeyName", "SecondKeyValue");
startActivity(myIntent);
```

- In order to get the parameters values inside the started activity, you must call the get[type]Extra() on the same intent:

```
// getIntent() is a method from the started activity
Intent myIntent = getIntent(); // gets the previously created intent
String firstKeyName = myIntent.getStringExtra("firstKeyName");
String secondKeyName= myIntent.getStringExtra("secondKeyName");

//If parameters are integers, use getIntExtra() instead etc.
```

Example: Navigation

```
public class MainActivity extends Activity {
    Button b1;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        b1 = (Button) findViewById(R.id.button);
        b1.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                Intent in=new Intent(MainActivity.this, SecondActivity.class);
                startActivity(in);
            }
        });
    }
}
```

…Cont'd

```
public class SecondActivity extends Activity {
    WebView wv;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main_activity2);
        wv = (WebView) findViewById(R.id.webView);
        wv.setWebViewClient(new MyBrowser());
        wv.getSettings().setLoadsImagesAutomatically(true);
        wv.getSettings().setJavaScriptEnabled(true);
        wv.loadUrl("http://www.google.com");
        }
        private class MyBrowser extends WebViewClient {
            @Override
            public boolean shouldOverrideUrlLoading(WebView view, String url) {
            view.loadUrl(url);
            return true;
        }
    }
}
```

- The content of **activity_main.xml**

```
<ImageView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:id="@+id/imageView"
    android:src="@drawable/logo"
    android:layout_centerHorizontal="true"
    android:theme="@style/Base.TextAppearance.AppCompat" />

<Button
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="First Page"
    android:id="@+id/button"
    android:layout_below="@+id/imageView" />
```

…Cont'd

- The content of **activity_second.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android=http://schemas.android.com/apk/res/android
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:weightSum="1">

<WebView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:id="@+id/webView"
    android:layout_gravity="center_horizontal"
    android:layout_weight="1.03" />
</LinearLayout>
```

…Cont'd

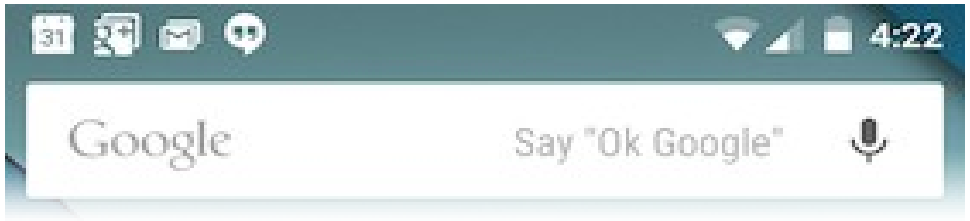
- The content of `AndroidManifest.xml File`

```
<application
        android:allowBackup="true"
        android:icon="@mipmap/ic_launcher"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:label="@string/app_name"
        android:supportsRtl="true"
        android:theme="@style/AppTheme">

        <activity android:name=".MainActivity">
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>

        <activity
            android:name=".SecondActivity"/>
    </application>
```

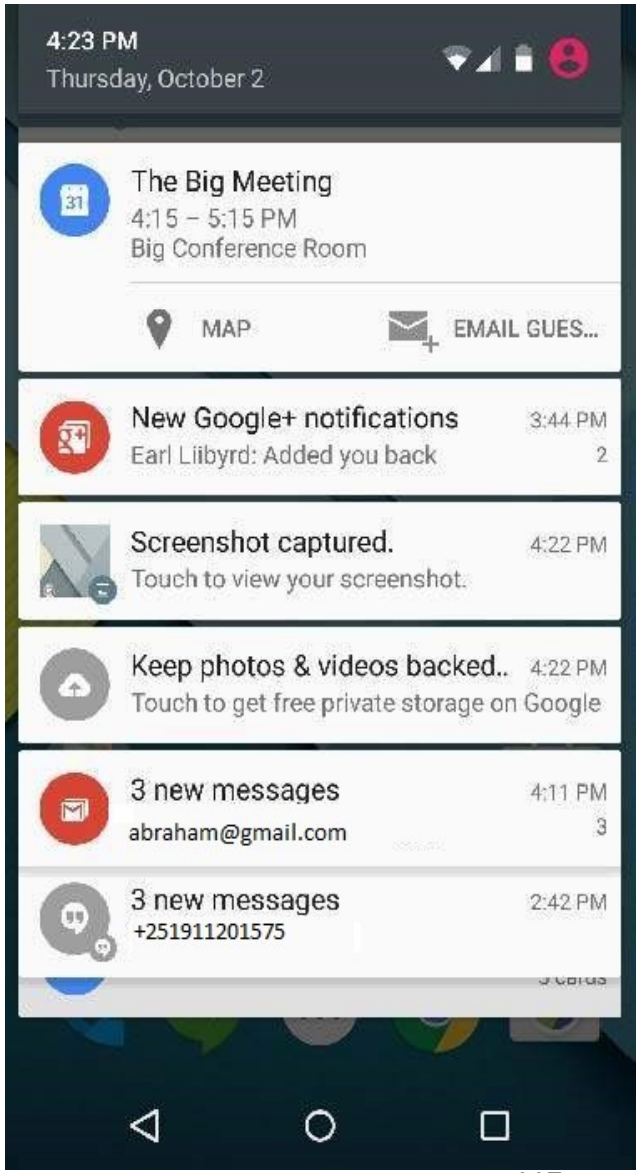
Notification with Status Bar and Toast

- Notification is a message that display to the user outside of app's normal UI.
- Notification first appears as an icon in the notification area.
- To see the details of the notification, the user opens the notification drawer.
- System-controlled areas that the user can view at any time:
  - Notification area and
  - Notification drawer
- Toast class provides a handy way to show users alerts but problem is that these alerts are not persistent which means alert flashes on the screen for a few seconds and then disappears.

…Cont'd



- To see the details of the notification, select the icon that display notification drawer having detail about the notification.
- On Emulator with virtual device, click and drag down the status bar to expand it detail.
- This will be just **64 dp** tall and called normal view.



- It can have Big View with additional detail about notification.
- Add up to six additional lines in the notification.
- The following screen shot shows such notification.

Create and Send Notifications

- Simple way to create a notification: Steps in app to create a notification –

Step 1 - Create Notification Builder

- Create a notification builder using **`NotificationCompat.Builder.build().`** Use **`Notification Builder`** to set various **`Notification`** properties (small and large icons, title, priority etc).

```
NotificationCompat.Builder mBuilder = new NotificationCompat.Builder(this)
```

Step 2 - Setting Notification Properties

- Once you have Builder object, set its Notification properties using Builder object as per the requirement.
- But this is mandatory to set at least following –
  - A small icon, set by **`setSmallIcon()`**
  - A title, set by **`setContentTitle()`**
  - Detail text, set by **`setContentText()`**

```
mBuilder.setSmallIcon(R.drawable.notification_icon);
mBuilder.setContentTitle("Notification Alert, Click Me!");
mBuilder.setContentText("Hi, This is Android Notification Detail!");
```

Step 3 - Attach Actions

- Optional part and required to attach an action with the notification.
- An action allows users to go directly from the notification to an **`Activity`** in the app, to look at one or more events or do further work.
- The action is defined by a **`PendingIntent`** containing an **`Intent`** that starts an **`Activity`** in the app.
- To associate the **`PendingIntent`** with a gesture, call the appropriate method of **`NotificationCompat.Builder.`** For example, to start **`Activity`** when the user clicks the notification text in the notification drawer, add the **`PendingIntent`** by calling **`setContentIntent().`**
- A **`PendingIntent`** object helps to perform an action on the app behalf, often at a later time, without caring of whether or not the app is running.
- Take help of stack builder object which contain an artificial back stack for the started Activity.
- This ensures that navigating backward from the **`Activity`** leads out of app to Home screen.

Example: Attach Actions

```
Intent resultIntent = new Intent(this, ResultActivity.class);
TaskStackBuilder stackBuilder = TaskStackBuilder.create(this);
stackBuilder.addParentStack(ResultActivity.class);

// Adds the Intent that starts the Activity to the top of the stack
stackBuilder.addNextIntent(resultIntent);
PendingIntent resultPendingIntent = stackBuilder
.getPendingIntent(0,PendingIntent.FLAG_UPDATE_CURRENT);
mBuilder.setContentIntent(resultPendingIntent);
```

Step 4 - Issue the Notification

- Finally, pass the Notification object to the system by calling **`NotificationManager.notify()`** to send the notification.
- Make sure to call **`NotificationCompat.Builder.build()`** method on builder object before notifying it.
- This method combines all of the options set and return a new Notification object.

```
NotificationManager mNotificationManager = (NotificationManager)
getSystemService(Context.NOTIFICATION_SERVICE);

// notificationID allows you to update the notification later on.
mNotificationManager.notify(notificationID, mBuilder.build());
```

The NotificationCompat.Builder Class

- The `NotificationCompat.Builder` class allows easier control over all the flags, as well as help constructing the typical notification layouts.
- The most frequently used methods available in `NotificationCompat.Builder` class are given below.

- The `NotificationCompat.Builder` Class

Constants	Description
`Notification build()`	Combine all of the options that have been set and return a new Notification object.
`NotificationCompat.Builder setAutoCancel (boolean autoCancel)`	Setting this flag will make it so the notification is automatically cancelled when the user clicks it in the panel.
`NotificationCompat.Builder setContent (RemoteViews views)`	Supply a custom RemoteViews to use instead of the standard one.
`NotificationCompat.Builder setContentInfo (CharSequence info)`	Set the large text at the right-hand side of the notification.
`NotificationCompat.Builder setContentIntent (PendingIntent intent)`	Supply a PendingIntent to send when the notification is clicked.
`NotificationCompat.Builder setContentText (CharSequence text)`	Set the text (second row) of the notification, in a standard notification.
`NotificationCompat.Builder setContentTitle (CharSequence title)`	Set the text (first row) of the notification, in a standard notification.
`NotificationCompat.Builder setDefaults (int defaults)`	Set the default notification options that will be used.

- The `NotificationCompat.Builder` Class

…Cont'd

Constants	Description
`NotificationCompat.Builder setLargeIcon (Bitmap icon)`	Set the large icon that is shown in the ticker and notification.
`NotificationCompat.Builder setNumber (int number)`	Set the large number at the right-hand side of the notification.
`NotificationCompat.Builder setOngoing (boolean ongoing)`	Set whether this is an ongoing notification.
`NotificationCompat.Builder setSmallIcon (int icon)`	Set the small icon to use in the notification layouts.
`NotificationCompat.Builder setStyle (NotificationCompat.Style style)`	Add a rich notification style to be applied at build time.
`NotificationCompat.Builder setTicker (CharSequence tickerText)`	Set the text that is displayed in the status bar when the notification first arrives.
`NotificationCompat.Builder setVibrate (long[] pattern)`	Set the vibration pattern to use.
`NotificationCompat.Builder setWhen (long when)`	Set the time that the event occurred. Notifications in the panel are sorted by this time.

Example: Notification

```
public class MainActivity extends Activity {
    Button b1;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState); setContentView(R.layout.activity_main);
        b1 = (Button)findViewById(R.id.button);
        b1.setOnClickListener(new View.OnClickListener() {
        @Override
        public void onClick(View v) { addNotification(); }
        });
    }
    private void addNotification() {
        NotificationCompat.Builder builder = new NotificationCompat.Builder(this)
        .setSmallIcon(R.drawable.logo)
        .setContentTitle("Notifications Example")
        .setContentText("This is a test notification");
        Intent notificationIntent = new Intent(this, MainActivity.class);
        PendingIntent contentIntent = PendingIntent.getActivity(this, 0, notificationIntent,
        PendingIntent.FLAG_UPDATE_CURRENT);
        builder.setContentIntent(contentIntent);
        // Add as notification
        NotificationManager manager = (NotificationManager) getSystemService(Context.NOTIFICATION_SERVICE);
        manager.notify(0, builder.build());
        }
}
```

…Cont'd

- The content of **res/layout/notification.xml** –

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent" >

<TextView
    android:layout_width="fill_parent"
    android:layout_height="400dp"
    android:text="Hi, Your Detailed Notification View goes here...." />
</LinearLayout>
```

- The content of **NotificationView.java**

```
public class NotificationView extends Activity{
    @Override
    public void onCreate(Bundle savedInstanceState){
        super.onCreate(savedInstanceState);
        setContentView(R.layout.notification)
    }
}
```

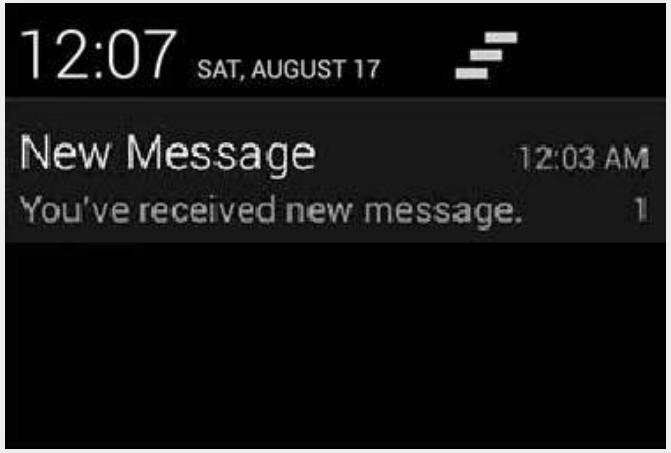
…Cont'd

- The content of **res/layout/activity_main.xml** –

```
<Button
    android:id="@+id/button"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Notification"/>
```

- The content of **AndroidManifest.xml** –

```
<activity android:name=".MainActivity"
    android:label="@string/app_name" >
    <intent-filter>
        <action android:name="android.intent.action.MAIN" />
        <category android:name="android.intent.category.LAUNCHER" />
    </intent-filter>
</activity>
<activity android:name=".NotificationView"
    android:label="Details of notification"
    android:parentActivityName=".MainActivity">
    <meta-data
    android:name="android.support.PARENT_ACTIVITY"
    android:value=".MainActivity"/>
</activity>
```



Localization

- Android application runs on devices in many different regions.
- To make app more interactive, it should handle text, numbers, files etc. in ways appropriate to the locales where application will be used.
- The way of changing string into different languages is called **localization.**

- **Localizing Strings**
  - To localize the strings in app , make a new folder under res with name of values-local where local would be the replaced with the region.
  - For example, in case of italy, make values-it folder under res.
  - Copy the strings.xml from default folder to values-it folder, change its contents. For example, changed value of hello_world string.

…Cont'd

- Italy, res/values-it/strings.xml

```
<;?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello_world">Ciao mondo!</string>
</resources>
```

- Spanish, res/values-it/strings.xml

```
<;?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello_world">Hola Mundo!</string>
</resources>
```

- French, res/values-it/strings.xml

```
<;?xml version="1.0" encoding="utf-8"?>
<resources>
    <string name="hello_world">Bonjour le monde!</string>
</resources>
```

…Cont'd

- The region code of other languages is given in the table below −

Language	Code
Afrikanns	Code: af. Folder name: values-af
Amharic	Code: am. Folder name: values-am
Arabic	Code: ar. Folder name: values-ar
Czech	Code: cs. Folder name: values-cs
Chinese	Code: zh. Folder name: values-zh
German	Code: de. Folder name: values-de
French	Code: fr. Folder name: values-fr
Japanese	Code: ja. Folder name: values-ja
Spanish	Code: es. Folder name: values-es

- Region Code of Languages for Localization

…Cont'd

- The content of xml res/layout/activity_main.xml:

```
<TextView
    android:id="@+id/textView"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text= “Debre Birhan University"
    android:textColor="#ff7aff24"
    android:textSize="35dp" />

<TextView
    android:id="@+id/textView2"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/amharic"
    android:textColor="#ff59ff1a"
    android:textSize="30dp" />
```

…Cont'd

- The content of xml res/layout/activity_main.xml.

```
<TextView
    android:id="@+id/textView3"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/french"
    android:textSize="30dp"
    android:textColor="#ff67ff1e"/>

<TextView
    android:id="@+id/textView4"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/arabic"
    android:textColor="#ff40ff08"
    android:textSize="30dp" />
```

- The content of xml res/layout/activity_main.xml.

```
<TextView
    android:id="@+id/textView5"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/chinese"
    android:textSize="30dp"
    android:textColor="#ff56ff12"/>
```

- The content of the res/values/string.xml.

```
<resources>
    <string name="app_name">Localization</string>
    <string name="hello_world">Hello World!</string>
    <string name="action_settings">Settings</string>
    <string name="amharic">አሶሳ ዩኒቨርሲቲ</string>
    <string name="french">Université d'Assosa</string>
    <string name="arabic">جامعة اسوسا</string>
    <string name="chinese">阿索萨大学</string>
</resources>
```

Context/Option Menu

- **Android Context Menu**
  - Context Menu
  - appears when user press long click on the element. It is also known as floating menu.
  - It affects the selected content while doing action on it.
  - It doesn't support item shortcuts and icons.

Example: Context Menu

- The content of xml `res/layout/activity_main.xml`.

```
<ListView
    android:id = "@+id/listView"
    android:layout_width = "368dp"
    android:layout_height = "495dp"
    android:textSize = "20dp"
    android:textColor = "#ff56ff12"
    android:layout_marginEnd = "8dp"
    android:layout_marginStart = "8dp"
    android:layout_marginTop = "8dp"
    app:layout_constraintEnd_toEndOf = "parent"
    app:layout_constraintHorizontal_bias = "0.0"
    app:layout_constraintStart_toStartOf = "parent"
    app:layout_constraintTop_toTopOf = "parent"/>
```

…Cont'd

- The content of xml `menu_main.xml`.

```
<menu xmlns:android ="http://schemas.android.com/apk/res/android">
    <item
          android:id = "@+id/call"
          android:title = "Call" />
    <item
          android:id = "@+id/sms"
          android:title = "SMS" />
</menu>
```

…Cont'd

- The content of xml `MainActivity.java`.

```
public class MainActivity extends AppCompatActivity {

    ListView listView;
    String contacts[] = {"Abraham", "Biniam", "Cheru", "Desta", "Elias"};

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        listView=(ListView)findViewById(R.id.listView);

        ArrayAdapter<String> adapter=new ArrayAdapter<String>(this,
        android.R.layout.simple_list_item_1,contacts);

        listView.setAdapter(adapter);

        // Register the ListView  for Context menu
        registerForContextMenu(listView);
    }
```

…Cont'd

```
    @Override
    public void onCreateContextMenu(ContextMenu menu, View v, ContextMenu.ContextMenuInfo menuInfo) {
        super.onCreateContextMenu(menu, v, menuInfo);

        MenuInflater inflater = getMenuInflater();
        inflater.inflate(R.menu.menu_main, menu);
        menu.setHeaderTitle("Select  Action");
    }

    @Override
    public boolean onContextItemSelected(MenuItem item){
        if(item.getItemId()==R.id.call){
            Toast.makeText(getApplicationContext(),"Calling Code", Toast.LENGTH_LONG).show();
        }
        else if(item.getItemId()==R.id.sms){
            Toast.makeText(getApplicationContext(),"Sending SMS Code", Toast.LENGTH_LONG).show();
        } else {
            return false;
        }
        return true;
    }
}
```

Android Option Menu

- Android `Option Menus` are the primary menus of android. They can be used for `settings`, `search`, `delete` item etc.
- Here, we are going to see two examples of option menus.
- First, the simple option menus and second, options menus with images.
- Here, we are inflating the menu by calling the `inflate()` method of `MenuInflater` class. To perform event handling on menu items, you need to override `onOptionsItemSelected()` method of `Activity` class.

Example: Option Menu

- The content of xml `res/layout/activity_main.xml`.

```
<android.support.design.widget.AppBarLayout
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:theme="@style/AppTheme.AppBarOverlay">

        <android.support.v7.widget.Toolbar
            android:id="@+id/toolbar"
            android:layout_width="match_parent"
            android:layout_height="?attr/actionBarSize"
            android:background="?attr/colorPrimary"
            app:popupTheme="@style/AppTheme.PopupOverlay" />

    </android.support.design.widget.AppBarLayout>

    <include layout="@layout/content_main" />
```

…Cont'd

- The content of xml `context_main.xml`.

```
<TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello World!"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintLeft_toLeftOf="parent"
        app:layout_constraintRight_toRightOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
```

- The content of xml `menu_main.xml`.

```
<menu xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    tools:context=".MainActivity">
        <item  android:id="@+id/item1"
               android:title="Item 1"/>
        <item  android:id="@+id/item2"
               android:title="Item 2"/>
        <item  android:id="@+id/item3"
               android:title="Item 3"
               app:showAsAction="withText"/>
</menu>
```

…Cont'd

- The content of xml MainActivity.java.

```
public class MainActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toolbar toolbar = (Toolbar) findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
    }

    @Override
    public boolean onCreateOptionsMenu(Menu menu) {
        // Inflate the menu; this adds items to the action bar if it is present.
        getMenuInflater().inflate(R.menu.menu_main, menu);
        return true;
    }
```

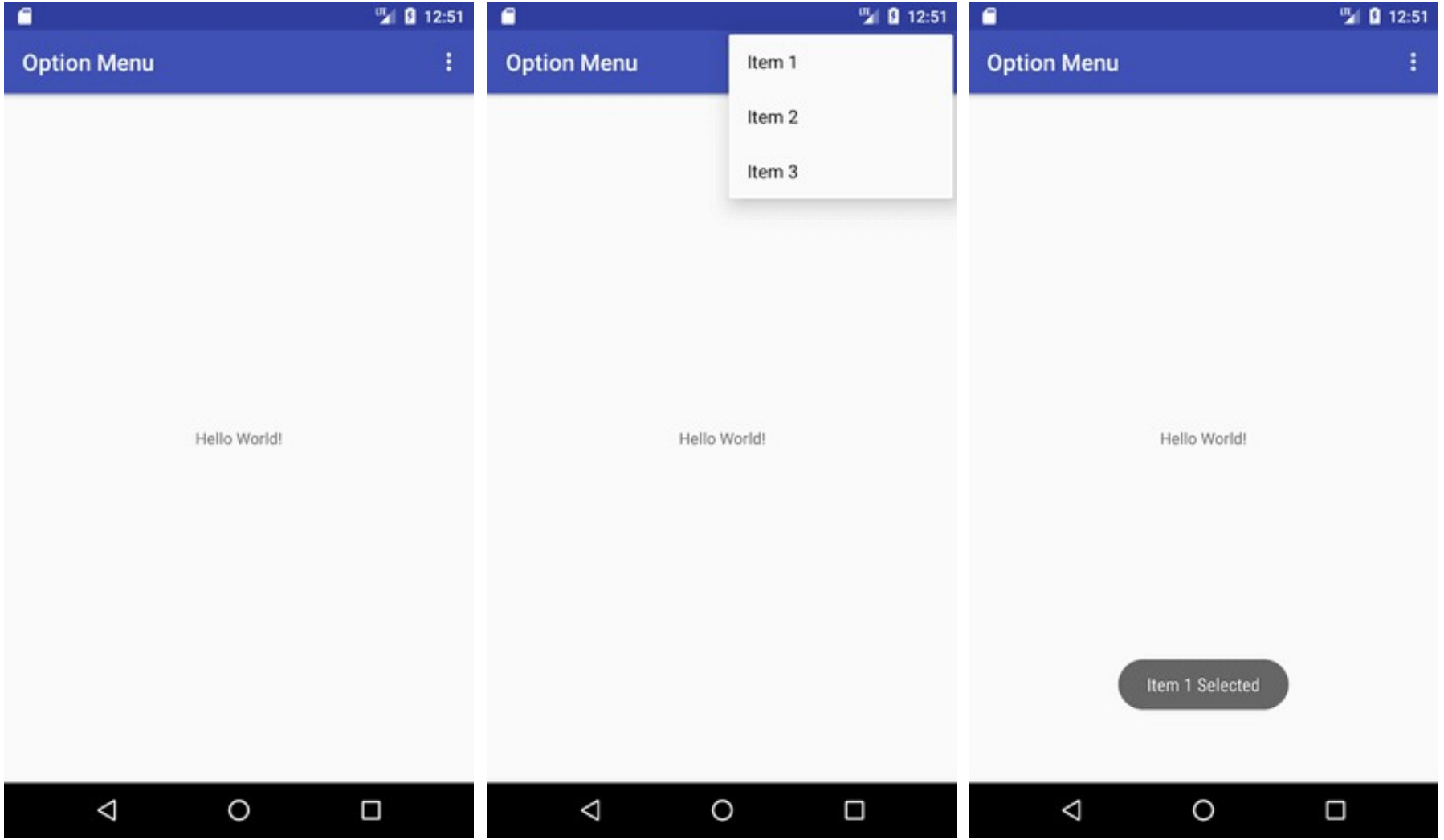
…Cont'd

- The content of xml `MainActivity.java`.

```
@Override
public boolean onOptionsItemSelected(MenuItem item) {
   int id = item.getItemId();
    switch (id){
        case R.id.item1:
            Toast.makeText(getApplicationContext(),"Item 1 Selected", Toast.LENGTH_LONG).show();
            return true;
        case R.id.item2:
            Toast.makeText(getApplicationContext(),"Item 2 Selected", Toast.LENGTH_LONG).show();
            return true;
        case R.id.item3:
            Toast.makeText(getApplicationContext(),"Item 3 Selected", Toast.LENGTH_LONG).show();
            return true;
        default:
            return super.onOptionsItemSelected(item);
    }
}
}
```

…Cont'd

- Output



Alert Dialog

- A Dialog is small window that prompts the user to a decision or enter additional information.
- Alert Dialog can used to ask user Yes or No in response of any particular action taken by remaining in same activity and without changing the screen
- To make an alert dialog, make an object of AlertDialogBuilder, inner class of AlertDialog.
- AlertDialogBuilder syntax is given below:

```
AlertDialog.Builder alertDialogBuilder = new AlertDialog.Builder(this);
```

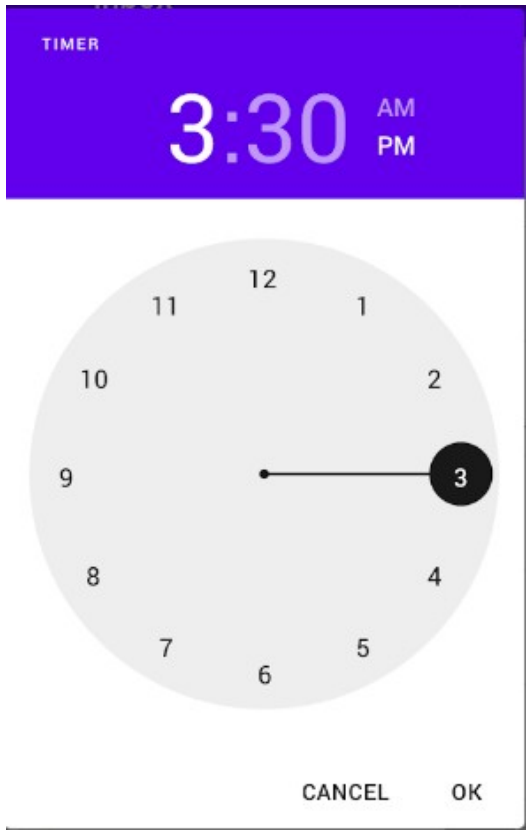
- Set Yes or No button using the object of the AlertDialogBuilder class.
- Syntax:

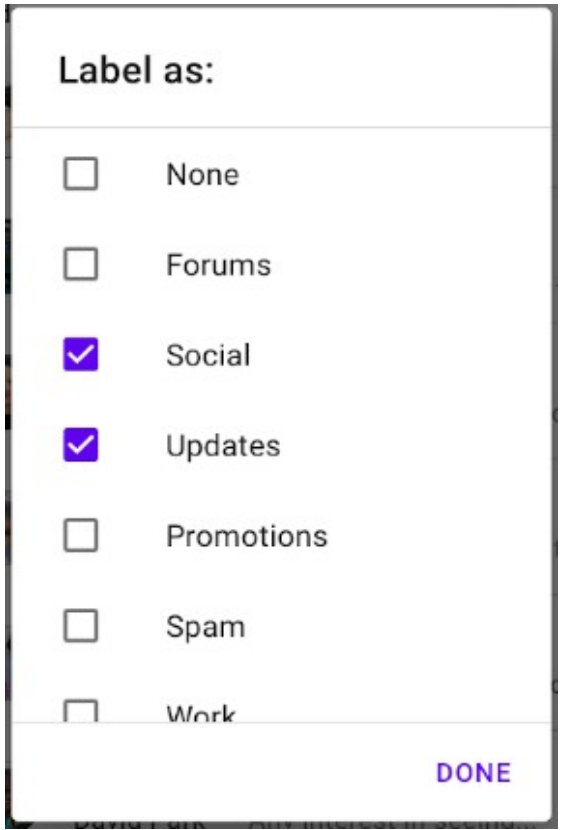
```
alertDialogBuilder.setPositiveButton(CharSequence text, DialogInterface.OnClickListener listener)
alertDialogBuilder.setNegativeButton(CharSequence text, DialogInterface.OnClickListener listener)
```

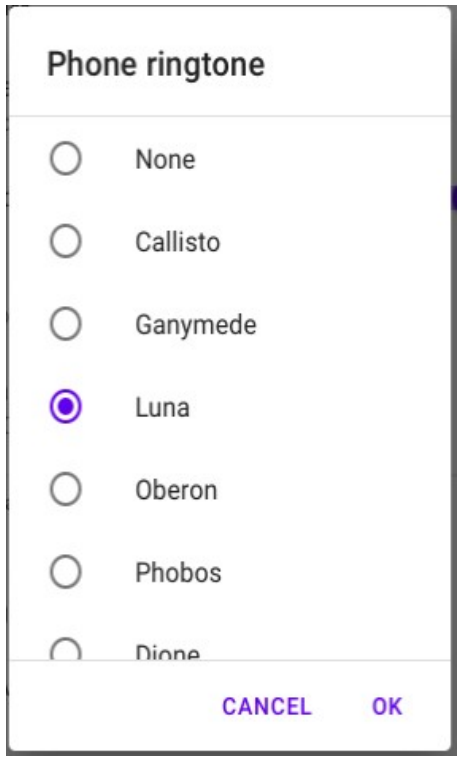
…Cont'd

- After creating and setting the dialog builder , create an alert dialog by calling the create() method of the builder class.
- Syntax:

```
AlertDialog alertDialog = alertDialogBuilder.create(); alertDialog.show();
```



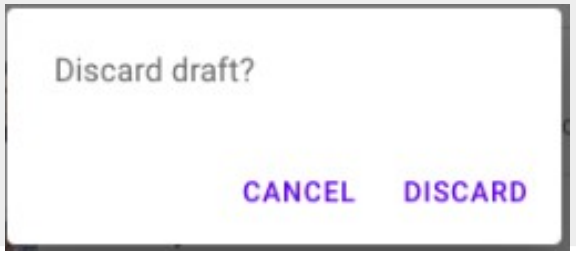




Dialog Fragment

- Dialog fragment.Dialog fragment is fragment that show fragment in dialog box.

```
public class DialogFragment extends DialogFragment {
    @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {
        // Use the Builder class for convenient dialog construction
        AlertDialog.Builder builder = new AlertDialog.Builder(getActivity());
        builder.setPositiveButton(R.string.fire, new DialogInterface.OnClickListener() {
            public void onClick(DialogInterface dialog, int id) {
                toast.makeText(this, "Enter a Text Here", Toast.LENTH_SHORT).show();
            }
        })
        .setNegativeButton(R.string.cancel, new DialogInterface.OnClickListener() {
            public void onClick(DialogInterface dialog, int id) {
                finish();
            });
            // Create the AlertDialog object and return it
            return builder.create();
        }
    }
}
```



❑ AlertDialog Example: MainActivity.java

```
public class MainActivity extends ActionBarActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
    public void open(View view){
        AlertDialog.Builder alertDialogBuilder = new AlertDialog.Builder(this);
        alertDialogBuilder.setMessage("Are you sure, You wanted to make decision");
        alertDialogBuilder.setPositiveButton("yes", new DialogInterface.OnClickListener() {
            @Override public void onClick(DialogInterface arg0, int arg1) {
                Toast.makeText(MainActivity.this,"You Clicked Yes Button",Toast.LENGTH_LONG).show();
            }
        });
        alertDialogBuilder.setNegativeButton("No",new DialogInterface.OnClickListener() {
            Override
            public void onClick(DialogInterface dialog, int which) {
                finish();
            }
    });
    AlertDialog alertDialog = alertDialogBuilder.create();
    alertDialog.show();
    }
}
```

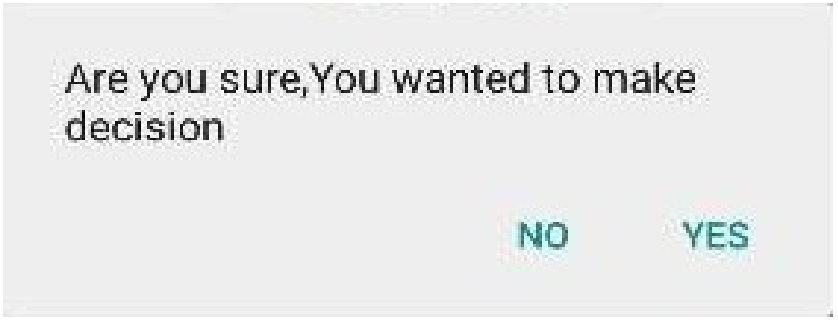
AlertDialog Example: activity_main.xml

```
<Button
    android:id="@+id/btnAlertDlg"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Alert Dialog"
    android:onClick="open"
    android:layout_alignLeft="@+id/imageView"
    android:layout_alignStart="@+id/imageView" />
```

❑ AlertDialog Example: Modify the res/values/string.xml to the following.

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
   <string name="app_name">AlertDialogExample</string>
</resources>
```

- Click on Yes: Toast Displays
- Click on No: finish() & close app



Thank You!

Questions